

Answer Key Evaluation DEP

Unified technical overview — one source file, one PDF

Project documentation deck

May 16, 2026

Outline (matches section order)

- ➊ **Introduction** — repository layout, system diagram, and how to extend the repo/deck.
- ➋ **Frontend** — React/Vite atlas and representative code.
- ➌ **Backend and APIs** — routers, REST surface, domain models, objective ingress paths.
- ➍ **Persistence and configuration** — SQLite, cache keys, Supabase pool, migrations, environment variables.
- ➎ **Authentication** — Supabase vs FastAPI boundary.
- ➏ **Gemini and Vertex** — how vision and text models are invoked (objective + code paths).
- ➐ **Objective grading pipeline (services)** — TikZ chain + module cheat-sheet (after Gemini overview).
- ➑ **Code evaluation** — C-from-image, gcc, syntax repair; then **LeetCode testcase library**.
- ➒ **Google Drive and Sheets** — OAuth scopes + roster detection hooks; full workbook read/write comes next.
- ➓ **ZIP, PDF, Sheets publishing, and Evaluate UI** — end-to-end faculty flow (ingress, tabs, export/sync) using those integrations.

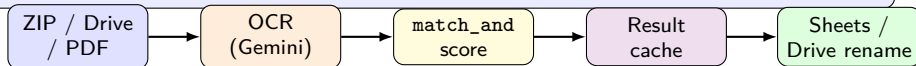
How to read this deck

- **Objective grading:** ZIP, Drive folder, or PDF → OCR → answer-key match → SQLite/cache → optional Sheets/Drive rename.
- **Code grading:** separate HTTP prefix `/api/code-eval/`; shares Gemini patterns but uses `c_code_evaluator` + LeetCode-shaped tests.
- **How to extend:** most sections below include a slide titled *How to extend: ...* with concrete file paths and a minimal pattern to copy.
- Later slides follow section order: persistence/config → auth → Gemini → objective pipeline → code-eval (including LeetCode) → Drive/Sheets (*thin*: scopes/detection) → ZIP/PDF + Sheets publishing + Evaluate UI (*thick*: faculty flow) → throughput/fuzzy/API contracts → OCR reference → supplemental coverage → closing.

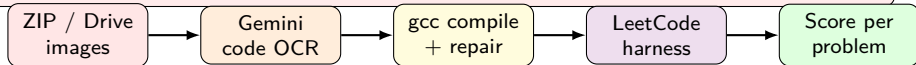
Two grading tracks: mental model

The repo supports two fully independent grading paths that share only Drive download utilities. Every section in this deck belongs to one or both tracks.

TRACK A — Objective grading (MCQ / numerical): `endpoints.py` + `batch_endpoints_optimized.py`



TRACK B — Code grading (handwritten C solutions): `code_eval_endpoints.py`



Track A

shared services: `drive_service`, `ocr_service` family, `evaluation_service`, `result_cache_service`, `sheets_service`.

Track B modules: `c_code_evaluator/evaluate_c_from_image.py`, `leetcode_mode.py`.

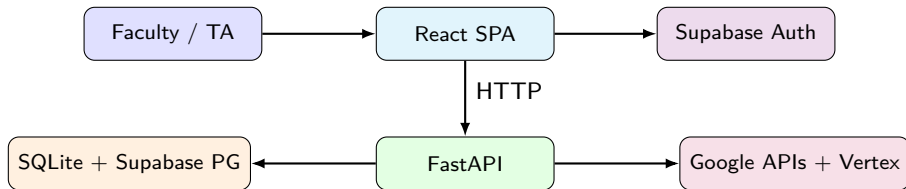
Where do I go to change X? (quick-lookup)

Goal	Primary files to touch
New JSON REST endpoint	backend/api/endpoints.py + model in backend/models.py
New high-throughput batch endpoint	backend/api/batch_endpoints_optimized.py; copy _handle_core pattern
New frontend page / route	src/pages/<name>/index.jsx, src/Routes.jsx, services/backendService.js
New Supabase column or table	backend/migrations/NNN_desc.sql + services/result_cache_service.py
New environment variable	Consume via os.getenv in backend/main.py; add to env checklist slide
New OCR region or model	Append entry to _ENDPOINT_POOL in batch_endpoints_optimized.py
New LeetCode problem/type support	Harness generators in c_code_evaluator/leetcode_mode.py
New answer-key file format	Parser branch in backend/services/answer_key_service.py
New CLI flag	backend/cli.py: parser.add_argument + branch in main()
New Drive API operation	Method on backend/services/drive_service.py
New Sheets tab or write pattern	backend/services/sheets_service.py
Tune fuzzy match thresholds	Constants in sheets_service.py: _smart_match_score, _name_similarity
Change Drive rename filename policy	Templates in backend/services/drive_rename_service.py
Change OCR image preprocessing	backend/services/image_preprocessing.py: preprocess_for_ocr kwargs
Add/change deploy secret (Fly)	fly secrets set KEY=value; mirror name in fly.toml comments

Top-level repository layout

Path	Purpose
<code>frontend-new-design/</code>	Vite React SPA: Supabase Auth + UI; calls FastAPI via <code>VITE_BACKEND_URL</code> .
<code>backend/</code>	FastAPI <code>main.py</code> ; <code>routers</code> <code>api/endpoints.py</code> , <code>batch_endpoints_optimized.py</code> , <code>code_eval_endpoints.py</code> .
<code>backend/services/*.py</code>	Drive, Sheets, OCR variants, evaluation, cache, PDF split, DB pool, etc.
<code>backend/migrations/*.sql</code>	Supabase DDL for <code>result_cache</code> , <code>pdf_page_map</code> .
<code>c_code_evaluator/</code>	Image→C code (Gemini); <code>compile-fix</code> ; LeetCode harness (<code>leetcode_mode.py</code>).

End-to-end system context (single product)



Same PDF covers UI wiring (left), API routers (center), persistence (cache + SQLite), vision/compiler backends (right), and the optional code-eval path.

How to extend: onboarding in this monorepo

New backend capability: add code under `backend/services/` or `backend/api/`, expose via router, document env vars next to existing checklist slides.

Example: a maintenance script at repo root → follow `extract_text_from_images.py` style (standalone if `__name__ == "__main__"` entrypoint calling shared services via `sys.path`).

Docs: each major section below ends with a slide titled *How to extend: ...* showing where and how to plug code in safely.

Quick start: what this section covers

Everything you need to get the project running locally and make a first change. Read this section once, then return to it whenever you set up a new machine.

Step	What you will do
1	Install system prerequisites (Python, Node.js, gcc)
2	Configure backend <code>.env</code>
3	Configure frontend <code>.env</code>
4	Start the backend server and verify <code>/api/status</code>
5	Start the frontend dev server
6	Apply Supabase SQL migrations
7	Run the smoke test

Prerequisites

Python 3.10+ Required for FastAPI backend. Verify: `python3 -version`.

Node.js 18+ Required for the Vite/React SPA. Verify: `node -version`.

gcc (system C compiler) Required for code-eval grading. macOS: `xcode-select --install`. Linux (Debian/Ubuntu): `sudo apt install build-essential`.

Google Cloud SA key Service-account JSON with **Drive + Sheets + Vertex AI** scopes. Download from GCP Console → IAM → Service Accounts → Keys → Add Key (JSON).

Supabase project Free tier is sufficient. Note the Project URL, anon key, and DB connection string (Settings → Database → Connection string → URI mode).

PyMuPDF (optional) Only needed for PDF ingress: `pip install pymupdf`. The PDF endpoint surfaces a clear `ImportError` hint if the package is absent.

Backend .env template (annotated)

```
# --- Google / Vertex (required for OCR, Drive, Sheets) ---
GOOGLE_APPLICATION_CREDENTIALS=/absolute/path/to/sa-key.json
GOOGLE_CLOUD_PROJECT=your-gcp-project-id
VERTEX_LOCATION=us-central1           # region for Vertex endpoints

# --- Gemini AI Studio (optional; fallback when no SA key) ---
GEMINI_API_KEY=AIza...                 # or GOOGLE_API_KEY

# --- Supabase cache (omit block -> local SQLite only) ---
CACHE_BACKEND=supabase
SUPABASE_DB_URL=postgres://postgres:password@db.xxx.supabase.co:5432/postgres
SUPABASE_POOL_MIN=1
SUPABASE_POOL_MAX=5

# --- Runtime tuning (optional; defaults shown) ---
PIPELINE_MAX_CONCURRENT=24             # lower on <1 GB RAM hosts
GEMINI_OCR_MODEL=gemini-2.5-flash-lite
CORS_ORIGINS=http://localhost:5173     # comma-separated list
ENABLE_LEGACY_PROCESS_FILE=0           # 1 enables deprecated endpoint
```

Frontend .env template

```
# frontend-new-design/.env.local
VITE_BACKEND_URL=http://localhost:8000    # no trailing slash
VITE_SUPABASE_URL=https://xxx.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOi...
```

Key constraint: VITE_BACKEND_URL must not have a trailing slash. The `api()` helper in `backendService.js` prepends this value to every `/api/...` path.

In production point VITE_BACKEND_URL to your deployed FastAPI domain (Fly, Railway, or custom).

Backend startup commands

```
# From repo root
cd backend
pip install -r requirements.txt

# Development server (auto-reload on file save)
uvicorn main:app --reload --port 8000

# First health check
curl http://localhost:8000/api/status

# Expected response includes:
#   "answer_key_loaded": false
#   "cache_stats": {...}
#   "environment": {...}
```

The lifespan hook warms the Supabase asyncpg pool on startup (if `CACHE_BACKEND=supabase`). Watch the terminal: a pool connection failure surfaces as a warning but does not abort startup — the backend falls back silently to SQLite.

Frontend startup commands

```
cd frontend-new-design
npm install          # installs node_modules (once per machine)
npm run dev
# Vite prints: VITE v5.x  ready at http://localhost:5173
```

First login: register a new account using the signup form at /login-register. The AuthProvider stores the Supabase JWT in localStorage under the key sb-{PROJECT_REF}-auth-token (set in supabaseClient.js).

Alternatively, create a user programmatically: `node frontend-new-design/create_test_user.js` (edit email/password at the top of that file).

Supabase migration checklist

Apply these SQL files in **Supabase SQL Editor** (Project → SQL Editor → New query → paste → Run):

- 1 `backend/migrations/001_result_cache.sql`
Creates `result_cache` table with JSONB columns + enables RLS.
- 2 `backend/migrations/002_cache_evaluation_scope.sql`
Adds `evaluation_id`, `file_id` columns + composite index to `result_cache`.
- 3 `backend/migrations/002_pdf_page_map.sql`
Creates `pdf_page_map` table for PDF reconciliation.

Naming collision: two files share the `002_` prefix — this is a known repo quirk. Both must be applied; they touch different tables so order between them does not matter.

After applying all three: set `CACHE_BACKEND=supabase` and restart the backend. `GET /api/status` should then report `"cache_backend": "supabase"`.

Smoke test: verifying your setup

```
# Automated (from backend/)
python smoke_test.py

# Or manually with curl, step by step:

# 1. Health check
curl http://localhost:8000/api/status

# 2. Load an answer key without any file (manual set)
curl -s -X POST http://localhost:8000/api/answer-key/set-manual \
  -H "Content-Type: application/json" \
  -d '{"answers": {"1": {"question_type": "SMCQ",
    "correct_answer": "A", "positive_marks": 1.0}}}'

# 3. Confirm it is loaded
curl http://localhost:8000/api/answer-key
# Expected: {"total_questions": 1, "answers": {"1": {...}}}

# 4. Clear it
curl -X DELETE http://localhost:8000/api/results/clear
```

Quick-start summary

After completing all steps you have:

- A running FastAPI backend at `localhost:8000` wired to Google Cloud and optionally Supabase.
- A running Vite dev server at `localhost:5173` authenticated via Supabase.
- Three Supabase tables (`result_cache`, `pdf_page_map`) ready for cache writes.
- A working smoke test baseline to compare against after any future change.

Next: the remaining sections document every module in depth. Use the *Where do I go to change X?* table (Introduction section) as your entry point whenever you know *what* you want to add but not *where*.

Frontend: what this section covers

The frontend is a **Vite + React SPA** that communicates with the FastAPI backend over plain HTTP and with Supabase directly for auth and roster data. This section gives a complete file atlas so you can locate any piece of UI code, understand how routing works, and follow the exact pattern for adding a new page or a new backend call.

Sub-topic	Slides
Extension pattern (new page + API call)	How to extend
Entry & routing files	File atlas — entry & routing
Service layer (API & Supabase calls)	File atlas — services
Page inventory	File atlas — pages
Bootstrapping, routing code	Code snippets
Supabase schema used by the frontend	Supabase application schema

How to extend: new screen + API call

Steps: (1) Create `src/pages/my-feature/index.jsx`; (2) add `<Route path="/my-feature" element={<ProtectedRoute><MyFeature /></ProtectedRoute>} />` in `Routes.jsx` with matching import; (3) add `async myPing()` in `backendService.js` using the same `api()` helper as existing calls.

Example pairing: frontend `return api('/my-widget')` $\leftrightarrow$ backend `@router.get("/my-widget")`
`def my_widget(): return { "ok": True }` under `/api` prefix.

Any batch-style action that needs cache isolation must pass a real `evaluation_id` (`requireEvaluationId`).

Frontend file atlas — entry & routing

File	Role
<code>src/index.jsx</code>	createRoot; mounts App; imports Tailwind/CSS.
<code>src/App.jsx</code>	Wraps AuthProvider around Routes.
<code>src/Routes.jsx</code>	BrowserRouter: public /login-register; protected /dashboard-overview, /faculty-dashboard, /course/:courseId, /evaluate/:evaluationId, /kanban-board, /task-detail, /analytics-dashboard.
<code>components/ProtectedRoute.jsx</code>	Spinner while useAuth loading; redirect if no user.
<code>components/ErrorBoundary.jsx</code>	Isolates render failures.
<code>components/ScrollToTop.jsx</code>	UX on navigation.
<code>components/ui/*</code>	Shared UI primitives (Header, Button, Input, modals).
<code>utils/cn.js</code>	Class-name merging helper.

Frontend file atlas — services

File	Role
<code>services/supabaseClient.js</code>	<code>createClient</code> ; <code>storageKey</code> per Supabase project ref; stale token cleanup.
<code>services/authService.js</code>	<code>signInWithPassword</code> , <code>signUp</code> , inserts profiles, friendly errors.
<code>services/backendService.js</code>	All FastAPI calls; <code>requireEvaluationId</code> ; multipart ZIP; batch URLs.
<code>services/evaluationService.js</code>	Supabase evaluations, duties, nested course/profile selects.
<code>services/resultsService.js</code>	Reads aggregated results for UI.
<code>services/courseService.js</code> , <code>studentService.js</code>	Course/student helpers.
<code>services/teamService.js</code> , <code>invitationService.js</code>	TA roster + invitations (used from course-detail, faculty-dashboard); Supabase-backed.

Frontend file atlas — pages (screens)

Area	Role
pages/login-register/* pages/dashboard-overview/*	Auth forms + social hooks. Landing tiles + quick actions; links target /sprint-planning, /team-management (see routing caveat).
pages/faculty-dashboard/* pages/course-detail/	Course hub + pending invitations. Course shell: Drive/Sheet URLs, evaluations, TA invites (teamService/invitationService).
pages/evaluate/index.jsx pages/kanban-board/*, pages/task-detail/*	Main grading: answer key, ZIP/Drive/PDF, export/sync/rename. Board + task detail shells.
pages/analytics-dashboard/ pages/sprint-planning/*, pages/team-management/*	Aggregate metrics. Code exists under src/pages/; not registered in Routes.jsx — dashboard/command palette links would hit catch-all redirect unless routes added.

Routes.jsx: mounted paths (authoritative)

```
<Route path="/login-register" element={<LoginRegister />} />
<Route path="/dashboard-overview" element={<ProtectedRoute><DashboardOverview /></ProtectedRoute>} />
<Route path="/faculty-dashboard" element={<ProtectedRoute><FacultyDashboard /></ProtectedRoute>} />
<Route path="/course/:courseId" element={<ProtectedRoute><CourseDetail /></ProtectedRoute>} />
<Route path="/evaluate/:evaluationId" element={<ProtectedRoute><EvaluatePage /></ProtectedRoute>} />
<Route path="/kanban-board" element={<ProtectedRoute><KanbanBoard /></ProtectedRoute>} />
<Route path="/task-detail" element={<ProtectedRoute><TaskDetail /></ProtectedRoute>} />
<Route path="/analytics-dashboard" element={<ProtectedRoute><AnalyticsDashboard /></ProtectedRoute>} />
```

Root / redirects to login; * → login. sprint-planning/ and team-management/ folders exist but have no Route here.

backendService.js: evaluation-scoped batch calls

```
const requireEvaluationId = (evaluationId, action) => {  
  if (!evaluationId || String(evaluationId).trim().toLowerCase() === 'default') {  
    throw new Error("Missing evaluationId for " + action);  
  }  
  return String(evaluationId).trim();  
};  
// processDriveFolder -> POST /api/batch/process-folder-optimized + evaluation_id
```

evaluate/index.jsx: answer key from Drive

```
const res = await backendService.extractAnswerKeyFromDrive(driveUrl);  
await evaluationService.saveAnswerKey(evaluation.id, res.answer_key);  
onKeyLoaded(res.answer_key);
```

Bootstrap: index.jsx and App.jsx

```
import { createRoot } from "react-dom/client";
createRoot(container).render(<App />);

import { AuthProvider } from "../context/AuthContext";
function App() {
  return (<AuthProvider><Routes /></AuthProvider>);
}
```

Dashboard overview and navigation affordances

dashboard-overview: metric-style landing + quick actions; navigates to sprint planning / team management paths *by URL only* — those routes are not declared in `Routes.jsx`.
`CommandPalette.jsx` registers keyboard-driven jumps to the same paths; useful after routes are wired.

Faculty dashboard and course detail

Faculty dashboard: course cards entry point; surfaces pending TA invitations via `invitationService`.

Course detail: binds Google Drive folder + master spreadsheet URLs to a course; evaluation list; TA search/add/remove and invite/cancel flows (`teamService`, `invitationService`).
Supabase rows drive persistence — FastAPI remains grading-only.

Kanban, task detail, analytics

Kanban board: column/card UI shell for duty-style tasks.

Task detail: attachments, comments, markdown notes, dependencies — PM metaphors adjacent to grading.

Analytics dashboard: read-mostly charts tied to evaluation throughput (expects backend results populated).

Supabase application schema (frontend-facing)

Not duplicated in backend/migrations SQL slides: React services expect tables such as profiles, courses, evaluations, duties/joins for TA invites. `evaluationService.saveAnswerKey` stores serialized keys per evaluation row; `courseService/studentService` wrap CRUD selectors. RLS and policies live in Supabase project — backend Postgres cache (`result_cache`) is a separate concern from app roster data.

Frontend: key takeaways

- **One API helper:** all backend calls go through `api()` in `backendService.js`. Never fetch `/api/*` directly from a component.
- **ProtectedRoute:** every authenticated page must be wrapped; a missing wrapper silently redirects to login.
- **Two data planes:** `backendService.js` talks to FastAPI; `evaluationService.js`, `courseService.js`, etc. talk directly to Supabase. Do not mix them in the same call.
- **evaluationId is required** for every batch action. Pass it from route params (`useParams()` in `evaluate/index.jsx`); the `requireEvaluationId` guard throws on 'default'.
- **Unregistered pages:** `sprint-planning/` and `team-management/` exist under `src/pages/` but are not in `Routes.jsx` — links from the dashboard will hit the catch-all redirect until you register them.
- **Next:** Backend and APIs shows the FastAPI side of every `backendService.js` call.

Backend and APIs: what this section covers

FastAPI is structured as **three routers** mounted in `main.py`. This section documents every endpoint group, the Pydantic request/response models, and the three objective ingress paths (ZIP, Drive, PDF) at a summary level. Deep-dives on specific flows (OCR, cache, Sheets, code-eval) appear in later sections.

Sub-topic	Slides
Extension pattern (new route)	How to extend
Backend file atlas (<code>main.py</code> , <code>models.py</code> , routers)	Backend file atlas
Endpoint groups (<code>/api/*</code> , <code>/api/batch/*</code>)	Endpoint group slides
Pydantic request/response contracts	<code>models.py</code> slides
How three routers compose	FastAPI composition

How to extend: new REST surface (endpoints.py)

Standard JSON route: add `@router.post("/my-action")` on `api/endpoints.py`; parse body with a Pydantic model from `models.py`; delegate to a service module; return dict or raise `HTTPException`.

Minimal example: `@router.get("/health-widget")` `async def health_widget(): return {"feature": "on"}` — already mounted via `main.py`.
High-throughput paths should follow `batch_endpoints_optimized.py`: `async aiohttp`, cache checks, semaphores — copy `_handle_core` patterns instead of blocking the event loop.

Backend file atlas

Path	Role
<code>main.py</code>	FastAPI app; CORS; lifespan (Supabase pool, OCR cleanup); includes 3 routers.
<code>models.py</code>	Pydantic: <code>AnswerKey</code> , <code>StudentResult</code> , <code>ProcessFolderRequest</code> , etc.
<code>database.py</code>	SQLite CRUD (students, submissions, results).
<code>cli.py</code>	Headless: Drive → OCR (local or Gemini) → score → Sheets.
<code>api/endpoints.py</code>	Monolithic standard API (ZIP, Drive, Sheets, cache, pipeline runs, rename).
<code>api/batch_endpoints_optimized.py</code>	Optimized batch + PDF runs + cache purge + reconciliation APIs.
<code>api/code_eval_endpoints.py</code>	Code grading HTTP surface; Zip + Drive image ingestion.

/api/* endpoints (representative groups)

- /status, /performance
- /answer-key (+ extract/upload/set-manual)
- /process-zip, /process-zip/start
- /process-drive-folder
- /export-to-sheets, /export-student-responses

- /full-pipeline*, /sync-*
- /results, /cache/*, /database/optimize
- /pipeline-runs/{id}/*
- /rename-drive-files*
- /download-answer-sheet-template

/api/batch/* and /api/pdf-runs/*

- POST /batch/process-folder-optimized (+ /start) — Drive throughput grading.
- GET /batch/processing-status/{id}, /batch/ocr-debug/{id}
- DELETE /cache/purge-evaluation/{evaluation_id}
- POST /batch/process-pdf-optimized* — PDF ingress.
- GET /pdf-runs/{id}/mapping, /mapping.csv, per-page image routes.
- POST /pdf-runs/{id}/reconcile
- Problem-image download helpers for QA.

/api/status vs /api/performance

GET /status: operational heartbeat + `answer_key_loaded`, in-memory results count, cache stats, SQLite analytics snapshot, runtime env flags.

GET /performance: same cache/DB analytics plus `optimization_recommendations` heuristics (cache size, pool hit ratio, batch ratio nudges).

/api/sync-drive and /api/scan-drive-folder

POST /sync-drive: query `folder_id` — legacy list-files only (images), returns raw file metadata count.

POST /scan-drive-folder: body `ProcessFolderRequest`; lists folder then `separate_files` into answer-key candidates vs student sheets — dry inspection before OCR spend.

Answer key REST surface (/api/answer-key*)

GET /answer-key — current in-memory key snapshot.

POST /answer-key/extract-from-drive — download + AnswerKeyService.extract_answer_key.

POST /answer-key/upload — multipart file upload path.

POST /answer-key/set-manual — JSON body installs key without file ingest.

full-pipeline: FullPipelineRequest (drive_folder_url, sheets_url) → awaits process_drive_folder then best-effort export_to_sheets; surfaces sheet_export_ok / error string without masking Drive success.

full-pipeline-with-rename: extends flow with Drive rename policy after grading (see endpoint docstring).

/api/sync-results

POST body: master sheet_url, optional results payload + subsheet_name.

Delegates to SheetsService.sync_results_with_master — reconciles OCR rows with roster ordering / merged marks after manual Sheet edits.

/api/results, cache hygiene, /api/database/optimize

GET /results: prefers session `_current_results`; falls back to SQLite legacy rows.

DELETE /results/clear: wipes session key + results.

POST /cache/cleanup|clear: LRU / destructive clears with guards.

POST /database/optimize: VACUUM, index repair via `optimized_db`; returns analytics snapshot.

/api/processing/status and pipeline run APIs

Legacy single-flight processing snapshot (distinct from batch `/batch/processing-status/{id}`).
`/pipeline-runs/{run_id}` (+ `/status`, `/items`) expose SQLite-backed ZIP/async runs for resume UX.

/api/process-file (legacy gate)

Disabled unless `ENABLE_LEGACY_PROCESS_FILE=1`. Otherwise HTTP 410 directing callers to Drive/ZIP/batch paths.

When enabled: downloads one Drive file, uses legacy `ocr_service.extract_data`, writes `submissions/results` — divergent schema from modern `StudentResult` pipeline.

Answer sheet template download

GET /download-answer-sheet-template? format=csv|xlsx|json & num_questions — emits blank structures describing SMCQ/MMCQ/NCQ fields for print workflows.

Objective ingress: ZIP, Drive, PDF

Three surfaces share `match_and_score` / cache semantics; differ only in how bytes reach FastAPI.

Where to read more: ZIP/PDF handlers, Sheets tabs, and Evaluate UI in **ZIP, PDF, Sheets publishing, and Evaluate UI**; Drive folder throughput (dual OCR, cache, rename) in **Throughput, fuzzy matching, and HTTP contracts**.

ZIP (/api/process-zip)

Multipart upload → extract images → optional answer key in archive → OCR → scoring → SQLite/cache → pipeline `run_id` for polling.

Drive (/batch/process-folder-optimized)

Folder ID → list/download → cache lookup `sheet:v2:{eval}:{file_id}` → dual Pro OCR + tie-break → batch writes → optional inline Drive rename.

PDF (/batch/process-pdf-optimized)

pdf_split_service renders pages → synthetic IDs pdf:<sha>:pN → same OCR+eval as ZIP → pdf_page_map + reconcile APIs.

Pydantic contracts (models.py)

```
class AnswerKeyEntry(BaseModel):
    question_type: str
    correct_answer: str
    positive_marks: float = 1.0
    negative_marks: float = 0.0

class StudentResult(BaseModel):
    entry_number: str
    name: str
    total_score: float
    max_score: float
    details: List[QuestionResult] = []
```

Requests (models.py)

```
class ProcessFolderRequest(BaseModel):
    folder_url: str
    evaluation_id: Optional[str] = None
    rename_drive_inline: bool = False
    group_multiple_pages: bool = False

class PipelineSummary(BaseModel):
    model_config = {"arbitrary_types_allowed": True}
    total_students_processed: int
    answer_key_source: str
    errors: List[Dict] = []
    processing_stats: Dict = {}
```

How objective routers relate (single backend)

`endpoints.py`: interactive ZIP/Drive/Sheets/templates/cache.

`batch_endpoints_optimized.py`: throughput + PDF world + purge.

`code_eval_endpoints.py`: separate C+LeetCode path — shares Drive download utilities only.

FastAPI composition (main.py)

```
app.include_router(endpoints.router)
app.include_router(batch_endpoints_optimized.router)
app.include_router(code_eval_endpoints.router)
```

Lifespan: warm asyncpg pool when `CACHE_BACKEND=supabase`; shutdown closes OCR clients + pool.

Backend and APIs: key takeaways

- **Three routers, three concerns:** `endpoints.py` (interactive/ZIP/Sheets), `batch_endpoints_optimized.py` (throughput/PDF/cache), `code_eval_endpoints.py` (code grading only).
- **Adding an endpoint:** JSON routes go in `endpoints.py`; high-throughput async paths follow `batch_endpoints_optimized.py` patterns (semaphores, aiohttp, cache-first).
- **Pydantic everywhere:** every request body and response must have a model in `models.py`. This enforces schema hygiene and generates OpenAPI docs automatically.
- **evaluation_id is a hard gate:** the batch router rejects blank or "default" values before any I/O — enforce this on all new evaluation-scoped endpoints.
- **Legacy gate:** `/api/process-file` is disabled by default; new code should never depend on it.
- **Next:** Persistence and configuration covers where results, cache rows, and pipeline state are actually stored.

Persistence and configuration: what this section covers

The backend uses **two persistence layers** and a **runtime configuration** layer. Understanding the boundary between them is essential before adding any new feature that stores data or reads configuration at deploy time.

Layer	Purpose
SQLite (WAL, local file)	Pipeline run state, per-file status, legacy student/submission rows. Always present; no credentials needed.
Supabase Postgres	<code>result_cache</code> JSONB (OCR + eval results); <code>pdf_page_map</code> . Active only when <code>CACHE_BACKEND=supabase</code> .
Environment variables	All runtime toggles, API keys, connection strings, tuning knobs.

Covered here: SQLite schema, WAL tuning, cache key format, Supabase pool, env var checklist, and the extension pattern for adding columns or tables.

How to extend: SQLite + Supabase persistence

Local WAL schema: extend `_ensure_schema / helpers` in `optimized_database_service.py`; use `CREATE TABLE IF NOT EXISTS` or guarded `ALTER TABLE`; add indexes for new query patterns.

Supabase: add `backend/migrations/003_your_change.sql`; apply in Supabase SQL editor; update `result_cache_service.py` read/write if columns or key semantics change.

Example: cache audit column `graded_by text DEFAULT NULL` → migration file + include field in upsert payload builder only when non-null.

SQLite schema (database.py)

```
CREATE TABLE IF NOT EXISTS students
(id TEXT PRIMARY KEY, name TEXT, roll_number TEXT)
CREATE TABLE IF NOT EXISTS submissions
(id TEXT PRIMARY KEY, student_id TEXT, exam_id TEXT,
file_id TEXT, status TEXT, extracted_data TEXT)
CREATE TABLE IF NOT EXISTS results
(submission_id TEXT PRIMARY KEY, score REAL,
feedback TEXT, details TEXT)
```

SQLite pipeline runs (optimized_database_service.py)

Beyond database.py legacy tables, WAL pool creates:

pipeline_runs: run_id, source_type (drive|zip|manual), source_ref, status counters, JSON errors.

pipeline_run_items: per-file status, optional ocr_json / result_json for resume & QA.

Local pdf_page_map table mirrors Supabase PDF mapping when runs originate from PDF splitter (same reconciliation semantics).

Sheet cache key (result_cache_service.py)

```
def sheet_cache_key(evaluation_id: str, file_id: str) -> str:  
    return f"sheet:v2:{evaluation_id}:{file_id}"
```

One row holds OCR JSON + optional scored result; answer_key_hash detects stale recompute when key edits.

SQLite WAL tuning (optimized_database_service.py)

```
conn.execute("PRAGMA_journal_mode=WAL")  
conn.execute("PRAGMA_synchronous=NORMAL")  
conn.execute("PRAGMA_mmap_size=268435456")
```

Supabase DDL (full excerpts later)

Row-level `result_cache JSONB` lives in Postgres when `CACHE_BACKEND=supabase`; complete `CREATE TABLE` and scope migration slides appear in **OCR and migrations reference**.

Supabase async pool (supabase_client.py)

```
1 _pool = await asyncpg.create_pool(  
2     dsn=os.getenv("SUPABASE_DB_URL"),  
3     min_size=int(os.getenv("SUPABASE_POOL_MIN", "1")),  
4     max_size=int(os.getenv("SUPABASE_POOL_MAX", "10")),  
5     statement_cache_size=0, # PgBouncer-friendly  
6 )
```

Environment variables (complete checklist)

Variable	Required?	Role and default
Frontend (prefix VITE_)		
VITE_BACKEND_URL	Yes	FastAPI base URL; no trailing slash
VITE_SUPABASE_URL	Yes	Supabase project URL
VITE_SUPABASE_ANON_KEY	Yes	Supabase public anon key
Google / Vertex (backend)		
GOOGLE_APPLICATION_CREDENTIALS	Yes*	Absolute path to SA JSON key (*required for Drive/Sheets/Vertex)
GOOGLE_CLOUD_PROJECT	Yes*	GCP project ID for Vertex AI endpoints
VERTEX_LOCATION	No	Vertex region; default us-central1
GOOGLE_API_KEY / GEMINI_API_KEY	No	AI Studio REST fallback; used if no SA key
GEMINI_OCR_MODEL	No	Model for header OCR; default gemini-2.5-flash-lite
Cache and database		
CACHE_BACKEND	No	sqlite (default) or supabase
SUPABASE_DB_URL	No*	Postgres URI (*required when CACHE_BACKEND=supabase)
SUPABASE_POOL_MIN	No	Async pool min size; default 1
SUPABASE_POOL_MAX	No	Async pool max size; default 10
Runtime tuning		
PIPELINE_MAX_CONCURRENT	No	Semaphore cap for simultaneous OCR+score; default 24
CORS_ORIGINS	No	Comma-separated allowed origins; default * (open)
ENABLE_LEGACY_PROCESS_FILE	No	1 re-enables deprecated /api/process-file; default 0

Persistence and configuration: key takeaways

- **SQLite is always on:** `optimized_database_service.py` creates WAL-mode tables at first run. You never need to provision it.
- **Supabase cache is opt-in:** set `CACHE_BACKEND=supabase + SUPABASE_DB_URL`; otherwise the backend uses in-memory LRU + SQLite.
- **Cache key format is sacred:** `sheet:v2:{evaluation_id}:{file_id}` — changing this format invalidates all existing cache rows. Only do it with a migration.
- **Answer key hash guards stale recompute:** when you edit an answer key, existing cache rows are purged automatically via `purge_stale_for_evaluation` on startup.
- **New Supabase column:** always use `ADD COLUMN IF NOT EXISTS` in migrations; always add a matching index if the column is used in `WHERE` clauses.
- **Next:** Authentication shows the Supabase JWT boundary between browser, FastAPI, and Supabase Postgres.

Authentication: what this section covers

The project has **two separate auth planes** that do not currently share tokens. Understanding this boundary is critical before adding any security-sensitive feature.

Plane	Who owns it
Browser ↔ Supabase	@supabase/supabase-js; JWT in localStorage; gates ProtectedRoute.
Browser ↔ FastAPI	Plain fetch; no JWT verification on most routes; relies on deployment-perimeter security.
Backend ↔ Google	Service-account JSON (GOOGLE_APPLICATION_CREDENTIALS); scoped per API.
Backend ↔ Supabase PG	SUPABASE_DB_URL DB role; bypasses row-level security.

Covered here: the current boundary, hardening path (JWT verification on FastAPI), and RLS audit considerations.

How to extend: auth-aware behavior

Today the SPA holds Supabase JWT/session; FastAPI routes largely trust network perimeter.

Frontend-only: gate screens with ProtectedRoute; read role or course from Supabase via evaluationService / AuthContext.

Backend-hardening example: add FastAPI dependency `def verify_jwt(authorization: str)` calling JWKS → attach `Depends(verify_jwt)` on destructive routes (`/cache/purge`, Drive rename); reject with 401 if missing.

Document new secrets (SUPABASE_JWT_SECRET or JWKS URL) next to env checklist slides.

Auth model (two planes)

Browser ↔ Supabase Email/password (or social) via @supabase/supabase-js. JWT/session stored under project-scoped storageKey. Rows in profiles, evaluations, courses — **not** validated by FastAPI today.

Browser ↔ FastAPI Plain fetch to /api/*; no Supabase JWT verification on backend for most routes — backend is “helpers + grading engines” behind your deployment perimeter.

Backend ↔ Google Service account JSON (GOOGLE_APPLICATION_CREDENTIALS) or API keys for Gemini REST.

Backend ↔ Supabase Postgres Optional SUPABASE_DB_URL when CACHE_BACKEND=supabase — DB role bypasses RLS.

Frontend: Supabase client + AuthProvider pattern

```
// supabaseClient.js - project ref isolates tokens
authOptions.storageKey = 'sb-' + PROJECT_REF + '-auth-token';

// App.jsx
<AuthProvider><Routes /></AuthProvider>

// ProtectedRoute.jsx - gate on useAuth().user
if (!user) return <Navigate to="/login-register" replace />;
```

Auth extrapolation

- Attach Supabase JWT to FastAPI (Authorization: Bearer) + verify with JWKS / shared secret.
- Map `profiles.role` to backend RBAC (who may purge cache, rename Drive, export Sheets).
- Audit log table keyed by Supabase auth subject id for institutional compliance.

Authentication: key takeaways

- **FastAPI is currently open on the network perimeter:** any caller who can reach the API can trigger grading, cache purge, or Drive rename. Secure the deployment host before exposing it publicly.
- **Hardening path:** add `Depends(verify_jwt)` on destructive routes; verify the Supabase JWT via JWKS URL or shared secret; reject with 401 if missing.
- **RLS note:** `SUPABASE_DB_URL` uses a Postgres role that bypasses RLS. The `result_cache` table has RLS enabled but no restrictive policies yet — add them before storing user-identifiable data.
- **New auth secret:** document every new secret (`SUPABASE_JWT_SECRET`, JWKS URL, etc.) next to the env var checklist slide.
- **Next:** Gemini and Vertex covers all AI/LLM call sites and which transport each one uses.

Gemini and Vertex: what this section covers

The repo makes Gemini/Vertex calls from **five distinct places** using two transports. Before adding a new AI call you must decide which transport to reuse and how to handle rate limits.

Transport	Auth	Used by
Vertex bearer	SA JSON + google.auth	ocr_service, batch OCR, evaluate_c_from_image
AI Studio REST	?key=GEMINI_API_KEY URL param	Fallback in ocr_service, evaluate_c_from_image, syntax-fix agent

Covered here: all call sites, why multiple clients exist, URL shapes, streaming vs non-streaming, and rate-limit retry patterns.

How to extend: new Gemini / Vertex call site

Reuse one existing transport: Vertex bearer token (`google.auth`) vs AI Studio URL with `?key=`.

Example: copy request JSON assembly from `ocr_service.py`; swap `model` string via env (pattern `GEMINI_OCR_MODEL`); keep `temperature` / `maxOutputTokens` explicit in payload.

For batch OCR throughput add another `{}` row to `_ENDPOINT_POOL` with realistic `rpm`; token bucket creates automatically on first use.

Gemini / Vertex: unified picture

Consumer	Transport	Typical env
OCRService, AnswerKeyService	Vertex SA bearer <i>or</i> AI Studio key URL	GOOGLE_APPLICATION_CREDENTIALS, GOOGLE_API_KEY
OptimizedOCRService / multi-region	Vertex regional generateContent	SA JSON per re- gion under sa-keys/ GOOGLE_CLOUD_PROJECT
Optimized batch pipeline evaluate_c_from_image.py	Policy: Gemini 2.5 Pro multi-region Vertex stream <i>then</i> fallback generativelanguage + key	Same SA pool; rate limiting internal GEMINI_API_KEY, VERTEX_LOCATION
code_eval_endpoints.py student header OCR	_vertex_stream_generate_content or REST key	GEMINI_OCR_MODEL default flash- lite
Syntax-fix agent	_vertex_generate_content _gemini_generate_content	or temperature 0

Why multiple Gemini “clients” in one repo?

- **Latency vs quality:** Flash/Lite for cheap/high-volume probes; Pro for disagreements in batch policy.
- **Auth flexibility:** On-prem SA JSON for Vertex; student laptops may only have AI Studio API keys for coursework demos.
- **Streaming vs non-stream:** Vision OCR uses stream aggregator; syntax-fix uses single-shot generate to simplify parsing.
- **Isolation:** Code-eval boots optional `vertex_key.json` and extends import path only inside `code_eval_endpoints`, leaving objective OCR configuration untouched.

Vertex streaming URL shape (evaluate_c_from_image.py)

```
url = (  
    f"https://{location}-aiplatform.googleapis.com/v1/"  
    f"projects/{project_id}/locations/{location}/"  
    f"publishers/google/models/{model}:streamGenerateContent "  
)  
headers = {"Authorization": f"Bearer_{creds.token}", ...}
```

AI Studio fallback: <https://generativelanguage.googleapis.com/v1beta/models/{model}:streamGenerateContent?key=...>

Rate limits: Gemini REST retries (`_gemini_stream_generate_content`)

```
for attempt in range(max_retries):  
    resp = requests.post(url, json=payload, timeout=timeout_s)  
    if resp.status_code == 429:  
        wait = 2 ** attempt * 5  
        time.sleep(wait)  
        continue
```

Gemini and Vertex: key takeaways

- **Pick the right transport:** if you have a service-account JSON, use the Vertex bearer path for production quality/quota. If running without SA (demo, student laptop), the AI Studio key URL is the fallback.
- **Never hardcode model strings:** use `os.environ.get("GEMINI_OCR_MODEL", "gemini-2.5-flash-lite")` as the pattern so ops can swap models without code changes.
- **Streaming vs non-stream:** vision OCR uses streaming (lower time-to-first-token for large images); syntax-fix and metadata extraction use single-shot `generateContent` for simpler response parsing.
- **Temperature matters:** set `temperature: 0.0` for structured extraction (JSON, C code) to keep outputs deterministic. Use higher values only for generative text.
- **Rate limits are handled:** `batch_endpoints_optimized.py` token buckets + 429 cooldown handle Vertex quota automatically. New call sites using Gemini REST must implement the exponential backoff pattern shown here.
- **Next:** Objective grading pipeline shows the full service chain that consumes these OCR results.

Objective pipeline: what this section covers

The objective grading pipeline is a **chain of service modules**, each with a single responsibility. This section maps the chain, describes every module's contract, and shows where to insert new logic.

Service	One-line contract
<code>drive_service</code>	List/download Drive files; separate answer-key vs student sheets.
<code>image_preprocessing</code>	Crop paper region; resize to safe Gemini payload.
<code>ocr_service</code> family	Send image to Gemini; return structured answer dict.
<code>evaluation_service</code>	Pure <code>match_and_score</code> : answer dict → <code>StudentResult</code> .
<code>result_cache_service</code>	Read/write cache row keyed by evaluation + file ID.
<code>sheets_service</code>	Write marks, response, and Super Sheet tabs.
<code>drive_rename_service</code>	Classify match confidence; rename Drive file.

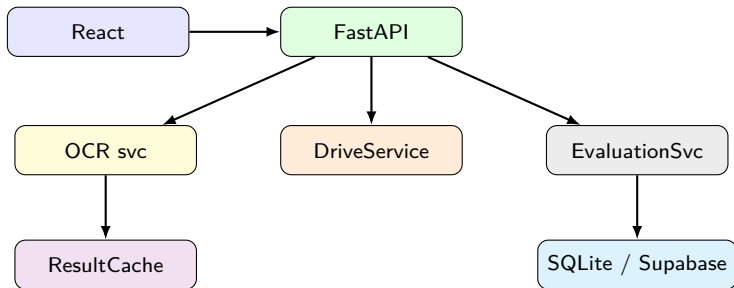
How to extend: objective pipeline hook

Add a pure helper in `backend/services/`; import from `batch_endpoints_optimized.py` inside `_handle_core` (after download, before/after OCR).

Example: extra deskew step → `await asyncio.to_thread(deskew_file, local_path)` to avoid blocking; write outputs back to same path Gemini reads.

Config knobs belong in env vars (`PIPELINE_MAX_CONCURRENT` precedent), not hard-coded constants, when ops teams must tune deploys.

Objective pipeline: responsibility chain



Batch optimized path: download bytes → preprocess → Gemini OCR (dual Pro + vote) → `match_and_score` → `cache row sheet:v2:{eval}:{file}` → optional Drive rename + Sheets export.

Service cheat-sheet (objective)

Module	Behavior
drive_service.py	OAuth SA or API key; list/download; batch rename.
pdf_split_service.py	PDF → page JPEGs; pdf:<sha>:pN synthetic IDs for cache stability.
image_preprocessing.py	Paper crop + JPEG sizing before model POST.
ocr_service.py	Legacy single-image Gemini Flash stream.
optimized_ocr_service.py /	Pooled HTTP; load-aware endpoint pick.
multi_region_ocr_service.py	
answer_key_service.py	Parse key files / vision-parse via Gemini.
evaluation_service.py	Pure match_and_score; SMCQ/MMCQ/NCQ + negatives.
batch_evaluation_service.py	Parallel scoring over many OCR dicts; stats.
result_cache_service.py	SQLite or Supabase; LRU; evaluation-scoped keys.
optimized_database_service.py	WAL SQLite pool; batch inserts.
sheets_service.py	Match entry regex; write totals / per-Q / comments.
drive_rename_service.py	Filename policy after grades (FUZZY-, UNMATCHED-, etc.).

Objective pipeline: key takeaways

- **Services are pure and composable:** `evaluation_service.match_and_score` has no I/O — pass it any answer dict and it returns a `StudentResult`. Test it in isolation without mocking.
- **Inject new steps around OCR:** add a preprocessing or post-processing step in `batch_endpoints_optimized.py` inside `_handle_core`, after download and before/after OCR. Use `asyncio.to_thread` for any CPU-bound work.
- **Cache is evaluation-scoped:** never write to the cache without an `evaluation_id`. Stale rows are auto-purged when the answer key changes.
- **Sheets and rename are optional exits:** the core pipeline (`OCR` → `score` → `cache`) is independent. Drive rename and Sheets export are called after the pipeline completes and can fail without invalidating grades.
- **Next:** Code evaluator deep dive covers Track B (C code grading), which reuses Drive download but is otherwise independent.

Code evaluator: what this section covers

The code evaluator lives entirely under `c_code_evaluator/` and `api/code_eval_endpoints.py`. It grades **handwritten or photographed C solutions** using a multi-stage pipeline.

Stage	What happens
OCR	Gemini Vision extracts C source from a photo with temperature 0.
Sanitize	Remove name/entry header lines that OCR picked up.
Compile	<code>gcc -c</code> compile-only pass as a fast syntax sanity check.
Repair	LLM syntax-fix agent (max 2 attempts); guarded by <code>validate_trivial_fix</code> .
Resolve	<code>leetcode_mode</code> : fetch test cases from file, pyleet, leetscrape, or LeetCode GraphQL.
Harness	Build a typed C main for each test case; full gcc link + run; compare output.
Score	Full marks only if all fetched test cases pass for a given problem slug.

How to extend: code evaluation surface

Add `@router.post(...)` in `code_eval_endpoints.py`; reuse `asyncio.to_thread(_process_single_image, ...)` for CPU/gcc work.

Answer-key JSON: extend problems map with a new question id $\rightarrow$ `{"3": { "slug": "new-slug", "marks": 5 }}`; ensure resolver returns cases (`cases_file` offline bundle if GraphQL blocked).

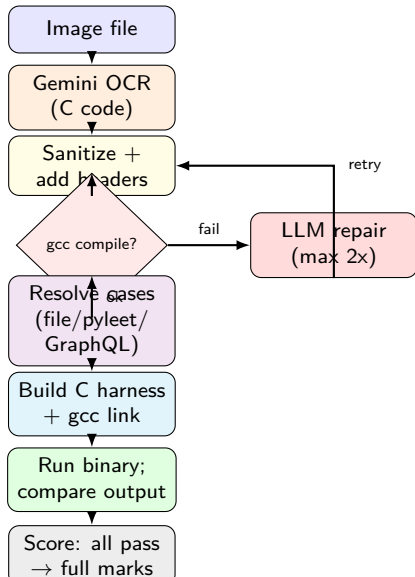
C evaluator changes live under `c_code_evaluator/`; keep `sys.path` manipulation localized to the router bootstrap already shown earlier.

Code evaluator: purpose

Grade **handwritten or photographed C solutions** per question using:

- Gemini/Vertex **vision OCR** to reconstruct C source from an image.
- **gcc** compile (-c object in FastAPI route; full link in leetcode_mode harness).
- Optional LLM **syntax repair** (max 2 attempts) with strict `validate_trivial_fix` guardrails.
- **LeetCode-shaped test cases**: fetched via `leetcode_mode` resolver `resolve_leetcode_problem_and_cases`, then executed in generated main harness per case.

Code evaluator: full pipeline flow



Evaluator bootstrap (code_eval_endpoints.py)

```
_vertex_key_path = str(Path(...) / "vertex_key.json")
if os.path.exists(_vertex_key_path):
    os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = _vertex_key_path
    os.environ["GOOGLE_CLOUD_PROJECT"] = ... # from JSON project_id

sys.path.insert(0, ... / "c_code_evaluator")
from evaluate_c_from_image import extract_c_code_from_image, ...
from leetcode_mode import resolve_leetcode_problem_and_cases, evaluate_leetcode_cases
```

Sets GOOGLE_CLOUD_PROJECT before importing modules that read it at import time.

HTTP surface (code_eval_endpoints.py)

```
router = APIRouter(prefix="/api/code-eval", tags=["code-evaluation"])

@router.post("/process-zip", response_model=CodeEvalResult)
async def process_zip_code_eval(file: UploadFile, answer_key: str = Form("")):
    # unzip images -> asyncio.to_thread(_process_single_image, ...)

@router.post("/process-drive-folder", response_model=CodeEvalResult)
async def process_drive_code_eval(request: DriveCodeEvalRequest):
    # DriveService.list_all_files_in_folder -> download -> same core processor
```

Answer key JSON for code eval

```
answer_key = {  
  "problems": {  
    "1": { "slug": "two-sum", "marks": 2 },  
    "2": { "slug": "valid-parentheses", "marks": 1 }  
  }  
}
```

slug is LeetCode titleSlug. Marks scale all-or-nothing per problem: all hidden examples must pass.

_process_single_image: step-by-step

- 1 **Student identity:** Gemini vision JSON extract (name, entry_number) with temperature 0; fallback regex from filename stem.
- 2 **Code OCR:** call `extract_c_code_from_image` with image path and API key — Vertex first if SA plus project, else API key.
- 3 **Sanitize:** drop header lines matching Name / Entry Number markers from OCR noise.
- 4 **Headers:** inject std includes if none present.
- 5 **Compile:** run GCC compile-only pass (`-c, -std=c11`).
- 6 **Repair loop:** up to two attempts of `fix_ocr_syntax_errors` guarded by `validate_trivial_fix`.
- 7 **Per problem:** `resolve_leetcode_problem_and_cases` then `evaluate_leetcode_cases`.
- 8 **Score:** full marks only when every fetched testcase passes for that slug.

Student metadata micro-OCR (code_eval_endpoints.py)

```
prompt = (  
    "Extract ONLY Name and Entry Number..."  
    "Return ONLY JSON with keys name and entry_number"  
)  
payload = {  
    "contents": [{"role": "user", "parts": [  
        {"text": prompt},  
        {"inline_data": {"mime_type": mime_type, "data": base64_image}},  
    ]}],  
    "generationConfig": {"temperature": 0.0, "maxOutputTokens": 256},  
}  
model = os.environ.get("GEMINI_OCR_MODEL", "gemini-2.5-flash-lite")
```

extract_c_code_from_image (evaluate_c_from_image.py)

```
prompt = (  
    "You are doing OCR on a photo of handwritten/printed C source code.\n"  
    "Return ONLY the reconstructed C code as plain text.\n"  
    "- Do NOT add explanations.\n"  
    "- Preserve newlines and indentation.\n"  
)  
payload = {  
    "generationConfig": {"temperature": 0.0, "maxOutputTokens": 2048},  
}  
# Vertex stream if GOOGLE_APPLICATION_CREDENTIALS + project else GEMINI_API_KEY  
return _strip_markdown_fences(raw)
```

fix_ocr_syntax_errors: LLM repair contract

```
prompt = (  
    "Fix ONLY trivial OCR-induced syntax errors..."  
    "Do NOT change logic, algorithms, or control flow.\n"  
    "ONLY fix: semicolons, brackets, OCR confusions 0/0, missing #include...\n"  
    "Return ONLY the corrected C code.\n"  
)  
# Uses _vertex_generate_content or _gemini_generate_content (non-stream)
```

validate_trivial_fix: safety rails

```
# Reject if >20 lines changed (SequenceMatcher opcodes)  
# Reject if for/while/switch/goto/do counts change  
# Reject disallowed new #includes (only stdlib whitelist)  
# Reject if operator token sequence drifts beyond small edit budget  
# Cap identifier rename drift (edit distance) bulk violations  
return True, "accepted"
```

Prevents “fix” pass from silently rewriting algorithms while repairing OCR typos.

ZIP ingestion implementation (process_zip_code_eval)

```
with tempfile.TemporaryDirectory() as tmpdir:
    zip_path.write_bytes(await file.read())
    zf.extractall(extract_dir)
    image_files = sorted([p for p in extract_dir.rglob("*")
                          if p.suffix.lower() in image_exts])
    for img in image_files:
        result = await asyncio.to_thread(
            _process_single_image, img, problems, work_dir, api_key)
```

`asyncio.to_thread` keeps FastAPI event loop responsive while CPU/gcc/Gemini-sync code runs in worker threads.

_compile_c in code-eval (syntax check)

```
1 proc = subprocess.run(  
2     ["gcc", "-c", str(source_path), "-O2", "-std=c11", "-Wall", "-Wextra",  
3     "-o", str(binary_path)],  
4     capture_output=True, text=True, timeout=timeout_s,  
5 )
```

Object file proves compilation; actual LeetCode grading uses separate full-program harness compile (leetcode_mode._compile_harness) linking generated main.

Response model (CodeEvalResult)

```
class CodeEvalResult(BaseModel):  
    total_students_processed: int  
    results: List[Dict[str, Any]]  
    errors: List[str]
```

Each result dict mirrors objective-sheet scoring fields: `total_score`, `details` with per-Q testcase counts, `comments` joining problem notes.

Dual gcc roles (why two compile paths?)

- **Endpoint _compile_c**: fast sanity check student OCR blob compiles as translation unit before wasting network on testcase fetch.
- **Harness compile**: each testcase wraps student functions + typed helpers + `main`; requires link-time symbols (`ListNode`, etc.).
- If sanity compile fails, repair agent runs; harness compile may still fail for unrelated reasons (unsupported signature).

How to extend: LeetCode resolver path

Add another adapter branch inside `resolve_leetcode_problem_and_cases` *before* GraphQL (preserve offline-first behavior).

Example: ship `cases_file` JSON next to exam ZIP — schema mirrors `leetcode_cases_sample.json`; resolver returns cases without network.

For new LeetCode-type (TreeNode helpers): extend harness generators in `leetcode_mode.py` where main wrapper is built; add golden row to fixture slides under supplemental coverage.

resolve_leetcode_problem_and_cases: merge strategy

- 1 Optional **JSON file**: `_parse_cases_file` — embed meta + explicit testcases for air-gapped runs.
- 2 Try **leetscrape family**: dynamic import `leetscrape` / `leetcode_scrape` / `leetscraper`; probe `get_problem`, `fetch_problem`, etc.
- 3 Try **pyleet**: `TestCaseRetriever` plus `get_testcase` helpers.
- 4 If still empty and slug set: **LeetCode GraphQL** `questionData` query — parse content HTML for Input/Output pairs + `exampleTestcases` fallback.
- 5 Drop cases without expected; `_dedupe_cases`; truncate to `max_cases`.

GraphQL probe (_fetch_from_graphql)

```
requests.post(  
    "https://leetcode.com/graphql",  
    json={"query": query, "variables": {"titleSlug": slug}},  
    timeout=timeout_s,  
)  
  
# question.metaData JSON -> LeetProblemSpec (function name, param types)  
# content HTML -> Input:/Output: regex pairs -> LeetCode args + expected
```

Network dependency: institution firewall may block; use cases_file fallback for exams.

Datamodels (leetcode_mode.py)

Type	Meaning
LeetProblemSpec	function_name, param specs (name + logical type string), return type, sources list.
LeetCodeCase	args list (Python literals after parsing), expected value, source provenance tag.
CFunctionSpec	Parsed C function signature from student OCR code via regex over bodies.

evaluate_leetcode_cases: execution model

- ① `extract_c_function_specs` on OCR code — find top-level functions; exclude `main`.
- ② `_select_function`: optional override, else match `problem.function_name`, else single candidate / last resort heuristic with notes.
- ③ For each case: `_render_case_harness` builds C file with helper structs for `ListNode/TreeNode`, binds args by type (`int`, arrays, `char*`, 2D grids), adds `main` that prints normalized output.
- ④ `gcc` full link harness → run binary → compare normalized stdout to expected string from `_serialize_expected`.
- ⑤ Aggregate `PASSED` / `WRONG_OUTPUT` / `TIMEOUT` / `COMPILE_ERROR` / `HARNESS_BUILD_ERROR`.

Harness selection excerpt (evaluate_leetcode_cases)

```
1 specs = extract_c_function_specs(code)
2 selected, selection_notes = _select_function(
3     specs,
4     override_name=function_name_override,
5     preferred_name=problem.function_name,
6 )
7 harness_code, expected_normalized = _render_case_harness(
8     code=code, func=selected, problem=problem, case=case)
9 _compile_harness(harness_path, harness_bin, timeout_s=compile_timeout_s)
```

Optional Python deps (grading machine)

- **pyleet** — programmatic official-style testcase tuples.
- **beautifulsoup4** — cleaner HTML strip for GraphQL content parsing (regex fallback if missing).
- **leetscrape** ecosystem (optional; multiple module names tried).
- **gcc** system compiler — required for any LeetCode harness run.

resolve_leetcode_problem_and_cases signature

```
def resolve_leetcode_problem_and_cases(  
    *, slug: Optional[str], question_id: Optional[str],  
    cases_file: Optional[Path], max_cases: int, timeout_s: int,  
) -> Tuple[LeetProblemSpec, List[LeetCase], List[str]]:  
    ...
```

Returns merged problem spec (function name + param metadata), deduped cases with expectations, and diagnostic notes strings from each fetcher.

JSON testcase file format (_parse_cases_file)

```
{
  "title_slug": "two-sum",
  "meta": { "name": "twoSum", "params": [...], "return": {...} },
  "testcases": [
    { "args": [[2,7,11,15], 9], "expected": [0, 1] },
    { "input": "...", "output": "..." }
  ]
}
```

pyleet path (_fetch_with_pyleet)

```
1 from pyleet import get_testcase
2 from pyleet.testcase_retriever import TestCaseRetriever
3
4 retriever = TestCaseRetriever()
5 question_data = retriever.get_question_data(resolved_slug)
6 meta = json.loads(question_data.get("metaData") or "{}")
7 pyleet_cases = get_testcase(title_slug=resolved_slug)
```

Builds LeetCode objects by adapting tuple inputs to arg lists based on parameter count inference.

leetscrape dynamic import (`_fetch_with_leetscrape`)

```
for module_name in ("leetscrape", "leetcode_scrape", "leetscraper"):
    module = importlib.import_module(module_name)
    # Try get_problem / fetch_problem / get_question ... with slug kwargs
```

Wide compatibility shim because PyPI package APIs differ; logs which module answered.

`_parse_example_pairs_from_content` (GraphQL HTML)

Regex scans stripped problem statement text for alternating `Input:` / `Output:` blocks; truncates at `Explanation:`.

Fallback: split `exampleTestcases` / `sampleTestCase` newline blocks into synthetic cases (may lack expected $\rightarrow$ dropped later).

Type-directed harness binding (`_build_input_binding`)

Maps Python literals from LeetCode cases to C declarations: `int` scalars, `bool`, C strings (`char buf[]`), `int` arrays (`int arr[] = {...}`), `char**` ragged strings, 2D grids with companion `colSizes`, linked lists via hidden `__build_listnode`, trees via `__build_tree` level-order arrays with null flags.

Printed output conventions

Integer scalars: `printf` with width-aware format (`%11d` for long long).

Booleans: strings `true/false`.

Arrays: bracketed comma-separated numbers.

ListNode/TreeNode: bracketed level-order / adjacency walk matching LeetCode aesthetics for string compare.

Case verdict aggregation (evaluate_leetcode_cases)

```
1 matched = case_run_ok and (not case_timed_out) and (  
2     actual_normalized == expected_normalized)  
3 if matched:  
4     status = "PASSED"; passed += 1  
5 elif case_timed_out:  
6     status = "TIMEOUT"  
7 elif not case_run_ok:  
8     status = "RUNTIME_ERROR"  
9 else:  
0     status = "WRONG_OUTPUT"
```

Utility script for generating diagnostic/error imagery around evaluator tooling (batch helpers). Not on the hot HTTP path but part of developer ergonomics for debugging OCR failures.

Code evaluator: key takeaways

- **gcc is a hard dependency:** if gcc is not on PATH, the entire code-eval path silently fails at the compile step. Verify with `gcc -version` after deployment.
- **Two compile stages:** the first gcc pass (`-c` object) is a cheap sanity check; the second (harness full link) is per test case. A failure in stage 1 triggers the repair agent; a failure in stage 2 is scored as `HARNESS_BUILD_ERROR`.
- **validate_trivial_fix is a safety rail:** it prevents the LLM from silently rewriting student algorithms. Do not disable it; instead widen the whitelist for new `#include` patterns if needed.
- **LeetCode network dependency:** GraphQL fetch is the last resolver and may be blocked by firewalls. Ship a `cases_file` JSON alongside exam archives for fully offline grading.
- **SA credentials are isolated:** `code_eval_endpoints.py` loads `vertex_key.json` from `c_code_evaluator/` separately. Changing it does not affect the objective OCR credentials.
- **Next:** Google Drive and Sheets covers the integration seams used by both tracks.

Google Drive and Sheets: what this section covers

This section covers the **integration seams**: OAuth scope requirements, service-account posture, and the column-detection logic that ties OCR output to roster rows. Full workbook roles (master / marks / response / Super Sheet), export endpoints, and the Evaluate page wiring appear in the next section.

Sub-topic	Slides
Extension pattern	How to extend
Drive scope + auth posture	<code>drive_service.py</code> scope slide
Sheets entry-number detection	ENTRY_NUMBER_PATTERN slide
Arc within the deck	Placement slide

Service-account JSON must include both `.../auth/drive` and `.../auth/spreadsheets` scopes. A common mistake is creating two separate SA keys; one key with both scopes is sufficient.

How to extend: Drive or Sheets integration

Drive: add a method on `DriveService` wrapping `googleapiclient` discovery; keep SA JSON path via `GOOGLE_APPLICATION_CREDENTIALS`; expose thin HTTP wrapper in `endpoints.py` only if UI needs it.

Example: `copy_file(src_id, dest_folder_id)` — build `files.copy` request body; handle 403 with actionable logs.

Sheets: extend `sheets_service.py`; new scopes → widen OAuth scope list + re-deploy credentials. Use `GET /api/sheets/preview` pattern for read-only probes before writes.

drive_service.py: Drive API scope

```
SCOPES = ['https://www.googleapis.com/auth/drive']  
# Service account JSON preferred; else GOOGLE_API_KEY for public folders only
```

sheets_service.py: entry detection

```
ENTRY_NUMBER_PATTERN = re.compile(  
    r'(\d{4})\s*([A-Za-z]{2,4})\s*(\d{2,5})',  
)  
# Header aliases for entry number, name, marks, comments columns
```

Drive and Sheets: how this section fits the arc

Here: **integration seams** — Drive scope, service-account posture, and how Sheets column/header detection (ENTRY_NUMBER_PATTERN) feeds roster alignment.

Next section: full **workbook roles** (master / marks / response / Super Sheet), export and preview endpoints, plus the Evaluate page wiring that calls them.

Later: Throughput deepens fuzzy matching, rename templates, and batch HTTP contracts that reuse the same services.

Google Drive and Sheets: key takeaways

- **One SA key, two scopes:** the service-account JSON needs both drive and spreadsheets scopes. Download from GCP Console; do not mix it with the Vertex AI key unless the project explicitly grants all three.
- **Entry-number pattern:** `ENTRY_NUMBER_PATTERN` must match your institution's roll format (year + branch code + serial). Edit the regex in `sheets_service.py` if your format differs.
- **Public folder mode:** without a SA key the Drive service falls back to an API key, which only works for publicly shared folders. Not suitable for student answer uploads — use SA key in production.
- **Adding a Drive operation:** wrap `googleapiclient.discovery.build("drive","v3",...)` calls inside a method on `DriveService`; handle `HttpError 403` (permission) and `404` (not found) explicitly.
- **Next:** ZIP, PDF, Sheets publishing, and Evaluate UI shows the full faculty workflow end-to-end, including all Sheets tabs created by the backend.

ZIP, PDF, Sheets, and Evaluate UI: what this section covers

This section documents the **complete faculty workflow** for one grading session: how student sheets enter the system, how results are written back to Google Sheets, and how the Evaluate page orchestrates it all.

Sub-topic	What you will find
Evaluate page	Route, layout, mode selector, toolbar actions
ZIP ingress	Frontend call, backend archive extraction, answer-key heuristics
PDF ingress	PyMuPDF page split, synthetic IDs, PDF batch orchestration
Sheets workbook	Master / marks / response / Super Sheet tab roles
Export pipeline	<code>/api/export-to-sheets</code> : three write steps, error capture
Student response tab	Per-question grid + answer-key row
Super Sheet	Longitudinal cumulative grades across evaluations

How to extend: Evaluate UI ingress or export

New upload mode: add `processingMode` branch in `evaluate/index.jsx`; duplicate `FormData.append('file', ...)` pattern from ZIP/PDF; call new `backendService.startFoo()` mirroring `startPdfProcessing`.

Example: multipart field names must match FastAPI `UploadFile` parameters (`file`, `evaluation_id` `Form`).

Export tweak: extend `handleExportToSheet` mapping of results before `exportToSheets` — follow existing `mappedResults` transformation for column hygiene.

Evaluate page: route and purpose

React route `/evaluate/:evaluationId`

(`frontend-new-design/src/pages/evaluate/index.jsx`) — faculty workflow for one quiz: load answer key, pick ingress (Drive / ZIP / PDF), run OCR pipeline, inspect results, sync/export to Google Sheets, optional Drive rename.

Evaluate UI: layout and panels

Responsive grid (1g:grid-cols-3): left column stacks **Answer Key** panel and **OCR Pipeline** card; wider column for results table, stats, and logs.

Objective vs **code** evaluations swap AnswerKeyPanel vs CodeAnswerKeyPanel.

Shared Tailwind tokens: bg-surface, border-border, text-text-primary, semantic success/error callouts.

Processing mode selector (frontend)

Segmented control with three modes:

Google Drive — folder URL input (`drive.google.com/drive/folders/...`).

ZIP Upload — dashed drop zone, `accept=".zip"`; expects answer key file + sheet images inside archive.

PDF — dashed drop zone `accept=".pdf"`; copy explains one student answer sheet per page, shared OCR/cache semantics with Drive.

ZIP and PDF: client calls (backendService.js)

- ZIP sync: FormData with file; optional query force_reprocess, extract_answer_key; POST /api/process-zip.
- ZIP async: POST /api/process-zip/start → run_id, poll /api/pipeline-runs/{id}/status.
- PDF async: FormData file + evaluation_id (required helper requireEvaluationId); POST /api/batch/process-pdf-optimized/start → processing_id, same poll as Drive batch.
- PDF sync: POST /api/batch/process-pdf-optimized → full PipelineSummary JSON.

Toolbar actions on Evaluate

Sync from Sheet — pull master roster alignment / refresh cues from Google Sheets URL tied to course.

Export to Sheet — POST export pipeline with subsheet + evaluation name; surfaces success message including `studentResponse_*` tab info when created.

Rename Drive files — batch rename after grading when folder URL present.

Live **pipeline log** lines mirror OCR progress (percent, errors, cache hits).

ZIP processing backend (endpoints.py)

POST /api/process-zip: save upload → `_process_zip_archive` extracts to temp tree → walk files into pseudo-Drive dicts (`id=local path`, `name`, `mimeType`, `local_path`).

`drive_service.separate_files` splits **answer key** candidates vs **student sheets** (filename heuristics).

Answer key: first match parsed via `AnswerKeyService`; `extract_answer_key=false` skips ZIP key if server already holds key.

Requires loaded answer key + at least one student sheet or returns 400/404 with actionable detail.

ZIP background starter

POST /api/process-zip/start: persists zip bytes → creates SQLite pipeline run
(source_type="zip", content-aware source_ref) → BackgroundTasks runs
_process_zip_archive_background.

Response: run_id, status_url for UI polling — mirrors long-running Drive batch ergonomics.

PDF split service (pdf_split_service.py)

PyMuPDF (fitz) renders each page to JPEG (default **220 DPI**, quality **90**) → balances checkbox legibility vs payload size.

Full-file SHA-256 pdf_hash for stable identity.

Each page becomes a synthetic “Drive-like” dict: `id = pdf:<hash>:p<n>`, `local_path`, `name`, `_pdf_meta` (`page_number`, `pdf_name`, `hash`).

Same shape lets `_process_sheets_optimized` consume PDF pages without a forked scorer.

PDF batch orchestration (batch_endpoints_optimized.py)

`_run_process_pdf_optimized`: status `splitting_pdf` → `split_pdf_to_pages` → assigns `processing_id`, persists rendered JPEG dir (`pdf_work_dir`) for later **page image** HTTP downloads. Then invokes the **same** optimized multicolumn OCR + cache + scoring path used for Drive folders (including `evaluation_id` scoping).
Failure surfaces in `_processing_stats` with `ImportError` hint if PyMuPDF missing.

- POST /api/batch/process-pdf-optimized/start — multipart file, optional evaluation_id form field; streams upload to temp with 1 MiB chunks; returns processing_id, status_url, mapping_url.
- POST /api/batch/process-pdf-optimized — synchronous; returns PipelineSummary or 500 on split/pipeline failure.
- Follow-on: /api/pdf-runs/{id}/mapping, .csv export (optional sheet_url for roster reconciliation), /reconcile, per-page image GET.

Google Sheets roles inside one spreadsheet

The product treats one Google Sheets URL as a **workbook hub**:

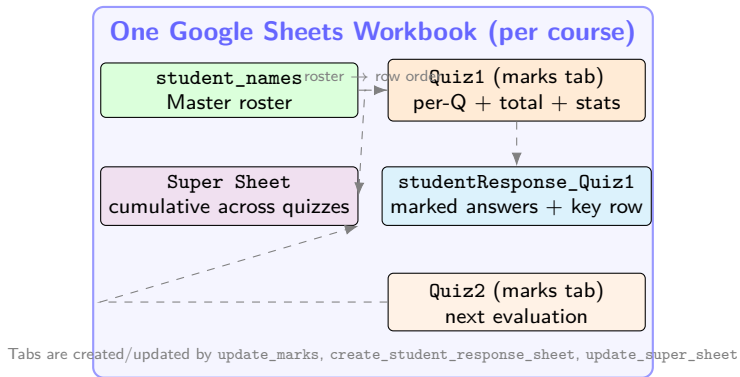
Master roster tab — canonical student rows (entry number + name); often named `student_names` when URL omits `#gid`.

Marks tab (per quiz) — created/updated by `update_marks`; per-question columns + total + comments + statistics block.

Student response tab — `studentResponse_{quiz}` with marked answers + answer-key row.

Super Sheet tab — one column per evaluation + cumulative total column for longitudinal grades.

Sheets workbook structure (visual)



Reading the master sheet (`read_student_list`)

Parses spreadsheet ID + optional gid/tab from URL.

If no tab in URL: prefer tab titled `student_names` (case-insensitive); else first sheet.

Reads grid via `values().get`; `_detect_columns` maps headers to entry / name / marks / comments aliases.

Builds `students` list with physical row indices for later writes.

Side effect: caches roster in `_master_students_cache` for rename fallback.

GET /api/sheets/preview

Non-destructive inspection: returns detected columns, chosen sheet name, full student list + count.
Frontend `backendService.previewSheet(sheetUrl)` powers validation before export/sync.

Writing marks (update_marks)

Reloads master students → smart-matches OCR results to roster rows → sorts rows **confident**, then **fuzzy**, then **unmatched**.

Emits wide table: master entry/name, OCR entry + corrected entry, OCR name, each question score column, SUM formula total column, merged comments (fuzzy labels + name mismatch warnings).

Appends **STATISTICS** block (mean, median, high, low via sheet formulas).

Applies conditional formatting bands for fuzzy/unmatched via `_format_marks_sheet`.

POST /api/export-to-sheets

Body: `ExportToSheetsRequest` (`sheet_url`, optional `results`, `answer_key`).

Query: `subsheet_name`, `evaluation_name` (choose marks tab title).

Resolution order for rows: request body `results` → in-memory `_current_results` → SQLite `students/results` legacy projection.

Steps: (1) `update_marks`; (2) if answer key + tab name — `create_student_response_sheet` as `studentResponse_{tab}`; (3) `update_super_sheet` column for this evaluation.

Returns aggregated summary JSON (per-step errors captured without failing entire export when optional steps fail).

Student response tab layout

`create_student_response_sheet`: columns Name, Entry Number, Q1..Qn from sorted answer-key ids; one row per student with marked text per question.

Trailing blank row then **Answer Key** row (USER_ENTERED); formatter bolds header + key row. Existing tab with same name is cleared then rewritten idempotently.

Super Sheet longitudinal column

Tab name fixed: Super Sheet.

If empty: seed headers Entry Number | Name | {evaluation} | Cumulative Total.

If populated: insert new evaluation column before Cumulative Total, shift cells, recompute row-wise cumulative sums so multiple quizzes accumulate without manual pivoting.

POST /api/export-student-responses

Dedicated path when only the detailed response grid is needed (still requires answer key).
Query `evaluation_name`; optional body overrides for `results` / `answer_key`.
Delegates to `export_responses_to_sheet` (service helper family alongside `create_student_response_sheet`).

Entry-number pattern (sheets_service)

ENTRY_NUMBER_PATTERN expects a four-digit year, a short branch code (letters), and a roll serial with flexible whitespace — used when auto-detecting which master-sheet column holds official entry numbers.

Design takeaway: one workbook, many surfaces

Faculty keep a **single** Spreadsheet ID in course metadata; the backend creates *tabs* for different consumers (roster vs per-quiz marks vs item analysis vs cohort totals). Separating concerns avoids copying CSVs manually while still letting accountants open one familiar Google Sheets URL.

ZIP, PDF, Sheets, and Evaluate UI: key takeaways

- **All three ingress paths converge:** ZIP, Drive folder, and PDF all produce the same “Drive-like file dict” shape before reaching the OCR pipeline. Adding a fourth ingress only requires producing that shape.
- **PDF needs PyMuPDF:** if the package is absent the endpoint raises a clear `ImportError`. Install `pymupdf` for any deployment that will handle PDF uploads.
- **Export order matters:** `/api/export-to-sheets` runs three steps in sequence (marks tab → student response tab → Super Sheet column). Each step can fail independently; errors are captured and returned without aborting prior successful steps.
- **Super Sheet is cumulative:** do not manually delete the Super Sheet tab between evaluations. The backend detects existing evaluations and inserts a new column; a fresh sheet loses history.
- **Subsheet naming:** the `subsheet_name` query parameter controls the marks tab title. Keep it consistent with `evaluation_name` so the Super Sheet and response tabs are aligned.
- **Next:** Throughput section covers the performance layer (rate limiting, dual OCR, fuzzy matching) that underlies the batch Drive path.

Throughput: what this section covers

This section explains how the batch Drive grading path achieves high throughput without overwhelming Vertex quotas, how student identities are matched to the roster despite OCR noise, and what all HTTP request/response shapes look like.

Sub-topic	Key mechanism
Load shaping	Token-bucket RPM limiter per Vertex region; 429 cooldown
Dual OCR + tiebreaker	Two independent Pro passes; majority vote on disagreement
Bounded concurrency	PIPELINE_MAX_CONCURRENT semaphore; <code>download_sem</code>
DB write pipeline	Async queue + batch SQLite writer (every 20 rows or 2s idle)
Two-phase retries	Phase 2 re-queues only failures with fresh OCR
Fuzzy matching	<code>_smart_match_score</code> : 5-tier entry + name similarity
Drive rename policy	Confident / FUZZY / UNMATCHED / NODATA templates
HTTP contracts	All Pydantic bodies, multipart shapes, and polling patterns

How to extend: throughput + fuzzy + contracts

More OCR quota: append to `_ENDPOINT_POOL` (model, region, rpm); redeploy → token buckets auto-scale spacing (60/rpm seconds).

RAM safety: set `PIPELINE_MAX_CONCURRENT` env lower on small VMs (documented near pipeline slides).

Fuzzy thresholds: adjust constants in `sheets_service.py` (`_smart_match_score`, `_name_similarity`) with unit snapshots + regression PDF exports.

HTTP schema: add fields to Pydantic models in `models.py`; bump FE TypeScript hand-written payloads only after backend accepts optional keys (backward compatible defaults).

Throughput section roadmap

Assumes you already saw ZIP/PDF ingress and Sheets publishing in the prior section; here we focus on **performance**, **roster fuzzy logic**, and **API shapes**.

- ➊ **Load shaping**: token buckets, round-robin regions, 429 cooldowns, dual OCR + tiebreaker (batch path).
- ➋ **Streaming pipeline**: bounded concurrency, DB writer queue, two-phase retries, cache tiers.
- ➌ **Fuzzy roster logic**: master-sheet vs OCR entry/name (sheets_service).
- ➍ **Rename policy**: confident vs fuzzy vs unmatched filenames (drive_rename_service).
- ➎ **HTTP contracts**: Pydantic bodies, multipart ZIP, polling vs synchronous batch.

Three OCR throughput backends (same Vertex idea, different orchestration)

Module	Role
multi_region_ocr_service.py	Many (region, model) endpoints (Flash / Flash-Lite / Pro); LoadBalancer.get_best_endpoint() scores load + latency + model priority + error rate.
optimized_ocr_service.py	Prefer healthy Flash-Lite with lowest load; fallback endpoints; retries with backoff.
batch_endpoints_optimized.py	Pro-only pool across regions; token bucket RPM ; round-robin ; dual OCR + third majority pass on disagreement.

Load balancer score (multi_region_ocr_service.LoadBalancer)

Lower combined score wins among healthy endpoints under capacity.

```
def score_endpoint(ep):  
    load_factor = ep.current_load / ep.max_concurrent  
    time_factor = ep.avg_response_time / 2.0  
    priority_factor = model_priority.get(ep.model, 4) / 4.0  
    error_rate = ep.total_errors / max(ep.total_requests, 1)  
    return load_factor + time_factor + priority_factor + error_rate
```

If all saturated: pick least-loaded healthy; if none healthy: error path.

Batch OCR endpoint pool (`_ENDPOINT_POOL`)

Nine `gemini-2.5-pro` Vertex regions with RPM budgets (US clusters ≈ 250 RPM each; EU smaller). Combined intent: $\sim 2,250+$ RPM headline capacity; stay on Pro for grading quality (no Flash fallback in this module).

Per-endpoint token bucket (`_TokenBucket.acquire`)

At most rpm tokens per 60s window: spacing $\approx 60.0 / \text{rpm}$ seconds between grants (lock-held schedule).

```
async def acquire(self):
    async with self._lock:
        now = time.monotonic()
        wait = max(0.0, self._next_allowed - now)
        self._next_allowed = max(self._next_allowed, now) + self.interval
    if wait > 0:
        await asyncio.sleep(wait)
```

Round-robin endpoint pick + 429 cooldown

`_pick_endpoint`: skip regions in exponential **cooldown** after HTTP 429; round-robin across remaining pool (global counter + lock).

Cooldown: base 30s, doubles per repeat hit on same region, max 300s; successes clear cooldown counters for that region.

Dual OCR and tiebreaker (batch)

- 1 Two independent Pro OCR passes on **different regions**.
- 2 If answers dicts agree on every question → accept.
- 3 Any disagreement → third pass on fresh region; **majority vote** per question.
- 4 Tiebreaker failure → majority of first two + warning tag in comments.

Streaming pipeline: bounded concurrency

PIPELINE_MAX_CONCURRENT (default 24): semaphore around download+OCR+score so hundreds of files do not spawn hundreds of concurrent coroutines (OOM on 512MB hosts).

download_sem: caps Drive downloads vs OCR slots ($\min(50, \max(8, \text{pipeline_max} * 2))$).

Background DB writer (pipelined commits)

Results enqueue to `asyncio.Queue`; worker flushes SQLite batches (every 20 rows or 2s idle); `None` sentinel ends run.

```
1 async def _db_writer_worker(queue, processing_id):
2     batch = []
3     while True:
4         item = await asyncio.wait_for(queue.get(), timeout=2.0)
5         if item is None:
6             if batch:
7                 await batch_write_student_results(batch, ...)
8                 break
9         batch.append(item)
10        if len(batch) >= 20:
11            await batch_write_student_results(batch, ...)
12            batch = []
```

Two-phase retry strategy (`_process_sheets_optimized`)

Phase 1: all sheets concurrent with bounded retries; empty entry / zero answers queue for phase 2 (does not block cohort).

Phase 2: retries only failures with **fresh OCR** (cache bypass), quieter network after bulk completes.

Cache tiers inside one cache row

Key pattern `sheet:v2:{evaluation_id}:{file_id}` stores JSON blobs:

Full eval hit Valid `evaluation_result`, sane entry, matching `answer_key_hash` → skip vision + scoring.

OCR-only hit Valid `ocr_result` → skip vision, still score.

Stale purge Startup `purge_stale_for_evaluation` removes rows whose hash differs from active key.

evaluation_id guard (cache isolation)

HTTP 400 if missing or literal default — prevents cross-quiz cache bleed.

```
if not evaluation_id or evaluation_id.strip().lower() == "default":  
    raise HTTPException(status_code=400,  
        detail="evaluation_id is required...")
```

Shared HTTP session limits (aiohttp)

One `ClientSession` per batch run with `TCPConnector(limit=256, limit_per_host=32, keepalive_timeout=60)` so concurrent Vertex POSTs reuse connections without exhausting file descriptors.

Fuzzy matching: where it applies

Sheets export correlates OCR rows with a master roster. `SheetsService._smart_match_score` outputs $[0, 1]$ confidence from normalized entry edits + OCR-aware name similarity; exporter writes commentary consumed by `DriveRenameService.classify_match`.

Entry normalization before comparison

Sheet + OCR entries pass `_correct_entry_number_ocr`, strip non-alphanumerics, uppercase — aligns formats like 2025 CSB 1454 vs OCR spacing noise.

Smart match stages (`_smart_match_score`)

- 1 Exact corrected entry: 1.0 if $\text{name_sim} \geq 0.40$ or OCR name empty; 0.98 if entry matches but names differ sharply.
- 2 Year slide: same branch+roll, year within ± 3 , $\text{name_sim} \geq 0.50 \rightarrow 0.95$.
- 3 Levenshtein ≤ 2 on corrected entries + $\text{name_sim} \geq 0.60 \rightarrow$ weighted blend capped 0.89.
- 4 Raw-entry Levenshtein ≤ 2 + $\text{name_sim} \geq 0.50 \rightarrow$ capped 0.88 when corrector mangled tokens.
- 5 Name-only rescue: $\text{name_sim} \geq 0.90 \rightarrow$ up to 0.85 when roll handwriting unusable.

Name similarity fusion (`_name_similarity`)

Word overlap + fuzzy per-word Levenshtein ($\sim 30\%$ edits), substring containment bonus, bigram Jaccard on despaced strings, `difflib.SequenceMatcher` baseline. Weighted sum (words/bigrams/sequence/containment) clipped at 1.0.

Rename classifier (classify_match)

Reads comments + entry:

fuzzy match → fuzzy; not found in master / absent → unmatched; empty/unknown/n/a entry → unmatched; else confident.

Drive filename templates

Category	Pattern
confident	{Entry} - {Name}.ext
fuzzy	FUZZY - {Entry} - {Name}.ext
unmatched (OCR text)	UNMATCHED - OCR {entry} - OCR {name}.ext
unmatched (no data)	UNMATCHED_NODATA - {stem}.ext
answer key	ANSWER_KEY - {original}.ext
error file	ERROR - {original}.ext

Duplicate entries append (DUP1), (DUP2) after sanitization.

Sanitization + skip rules

Replace unsafe filename chars; collapse whitespace; truncate length (50 chars default for student tokens).

Optional `skip_already_renamed`: regex detects known prefixes (FUZZY - , UNMATCHED, ANSWER_KEY - , ERROR - , roll-like prefix) to avoid rename churn on re-runs.

Inline rename during batch (`_rename_one`)

When `rename_drive_inline=True`: classify + compute target filename under async lock (duplicate counters); Drive API rename via `asyncio.to_thread`; successful rename updates `file_name/file_id` on the result object for exports.

RenameDriveFilesRequest

```
class RenameDriveFilesRequest(BaseModel):  
    folder_url: str  
    dry_run: bool = False  
    skip_already_renamed: bool = False  
    results: Optional[List[Dict]] = None  
    sheet_url: Optional[str] = None
```

POST /api/rename-drive-files; dry_run returns plan via execute_rename without mutating Drive.

ProcessFolderRequest

```
class ProcessFolderRequest(BaseModel):  
    folder_url: str  
    evaluation_id: Optional[str] = None  
    rename_drive_inline: bool = False  
    group_multiple_pages: bool = False
```

Powers `/api/batch/process-folder-optimized (+ /start)`; server rejects blank/default evaluation ids before work begins.

PipelineSummary

```
class PipelineSummary(BaseModel):  
    total_students_processed: int  
    answer_key_source: str  
    results: List[Any] = []  
    errors: List[Dict] = []  
    processing_stats: Dict = {}
```

Typical synchronous batch completion payload; starters return ids for polling endpoints instead.

Student row schema (StudentResult)

Identity (entry_number, name); aggregates (total_score, max_score, counts); negative_deduction; details list of QuestionResult (marked, correct, result, score); comments carries fuzzy/absent markers; file_name/file_id bind Drive assets.

AnswerKey + AnswerKeyEntry

```
class AnswerKey(BaseModel):
    total_questions: int
    answers: Dict[int, AnswerKeyEntry]
    negative_marking: float = 0.0
    metadata: Dict = {}

class AnswerKeyEntry(BaseModel):
    question_type: str # SMCQ, MMCQ, NCQ
    correct_answer: str
    positive_marks: float = 1.0
    negative_marks: float = 0.0
```

Sheets export models

```
class ExportToSheetsRequest(BaseModel):
    sheet_url: str
    results: Optional[List[Dict]] = None
    answer_key: Optional[Dict] = None

class SheetUpdateSummary(BaseModel):
    updated: int
    not_found: List[str] = []
    name_mismatches: List[Dict] = []
    errors: List[str] = []
```

Full pipeline models

```
class FullPipelineRequest(BaseModel):  
    drive_folder_url: str  
    sheets_url: str  
  
class FullPipelineWithRenameRequest(BaseModel):  
    drive_folder_url: str  
    sheets_url: str  
    rename_files: bool = True  
    dry_run_rename: bool = False
```

Code-eval HTTP models

```
class CodeEvalRequest(BaseModel):  
    answer_key: Dict[str, Any]  
  
class DriveCodeEvalRequest(BaseModel):  
    folder_url: str  
    answer_key: Dict[str, Any]  
  
class CodeEvalResult(BaseModel):  
    total_students_processed: int  
    results: List[Dict[str, Any]]  
    errors: List[str]
```

Multipart vs JSON FastAPI styles

JSON Body models (`ProcessFolderRequest`, `RenameDriveFilesRequest`, `DriveCodeEvalRequest`) with `application/json`.

Multipart ZIP uploads (`UploadFile`) + optional form fields (`answer_key` string).

Query Flags such as `force_reprocess`, `destructive_confirm=true`, cache filters.

Paths `/pipeline-runs/{run_id}`, `/pdf-runs/{id}/mapping`, etc.

Synchronous batch vs background starter

POST /api/batch/process-folder-optimized awaits full pipeline → immediate PipelineSummary.
POST .../start enqueues BackgroundTasks; HTTP returns processing_id.
Poll GET /api/batch/processing-status/{id} for processed counts, cache counters, error tails, partial snapshots.

Representative JSON envelope

Mirrors PipelineSummary: counts, provenance string, results arrays (serialized StudentResult), structured errors, processing_stats (cache_hits, ocr_cache_hits, timing fields).

Failure paths raise HTTPException(detail=...); validation errors include field paths via FastAPI/Pydantic.

Cache DELETE endpoints (examples)

- DELETE /api/cache/purge-evaluation/{evaluation_id}
- DELETE /api/batch/purge-stale-eval-cache
- POST /api/cache/cleanup, POST /api/cache/clear (legacy/global helpers with guards)

PDF run contracts (overview)

Multipart starter → processing_id;
GET /api/pdf-runs/{id}/mapping + CSV export;
POST /api/pdf-runs/{id}/reconcile;
binary page asset routes for QA viewers.

Optimization cheat-sheet (objective batch)

- Token buckets smooth Vertex quota usage ahead of 429 storms.
- Dual OCR + majority vote reduces single-model hallucinations.
- Semaphores cap RAM/FD on shared workers.
- Queue-backed SQLite writer amortizes disk writes during bursts.
- Full-eval cache avoids repeated vision when inputs unchanged.
- Phase-2 retries isolate pathological sheets without blocking everyone else.

Throughput: key takeaways

- **Token buckets are automatic:** adding a new entry to `_ENDPOINT_POOL` is all you need to expand quota. The bucket interval ($60.0/\text{rpm}$) is computed at registration time.
- **PIPELINE_MAX_CONCURRENT is the RAM knob:** on a 512 MB host set it to 4–8. On a 4 GB host the default 24 is safe. Always tune before deploying to a new tier.
- **Dual OCR is intentional overhead:** two Vertex Pro calls per sheet sounds expensive but eliminates the most common single-model hallucinations (answer A vs D confusion). Disable it only if cost is prohibitive.
- **Fuzzy match is a last resort:** if your institution's entry-number OCR quality is high, most matches will be exact or year-slide. Tune the 5-tier thresholds only if you see many false FUZZY labels.
- **Dry-run rename before committing:** `dry_run=true` in `RenameDriveFilesRequest` returns the planned filenames without mutating Drive. Always run this first on a new exam folder.
- **Next:** OCR and migrations reference is a compact operator reference for tuning preprocessor settings and managing Supabase DDL.

OCR and migrations: what this section covers

This is a **compact operator reference** for the parts of the system that run on every grading job. The performance deep-dive (token buckets, dual OCR policy) lives in the Throughput section; this section adds the module map, preprocessor signature, and full Supabase DDL excerpts.

Sub-topic	Slides
OCR module map	All OCR-related modules and their roles
001_result_cache.sql DDL	Full CREATE TABLE with column types
002 scope migration	evaluation_id, file_id, index
002 PDF page map	pdf_page_map DDL
SQLite vs Supabase boundary	Summary table
OCR preprocessor signature	preprocess_for_ocr tuning knobs

OCR and migrations: placement in this deck

You already saw batch OCR policy and pipeline knobs under **Throughput**; this section is a **compact operator reference**: module map, preprocessor signature, Supabase DDL excerpts, and SQLite vs cloud cache boundaries.

How to extend: OCR tuning + SQL migrations

Preprocessor: change defaults in `image_preprocessing.preprocess_for_ocr` or call-site kwargs inside batch download path; verify Gemini payload sizes stay under limits.

New Supabase migration: copy naming `NNN_description.sql`; include `IF NOT EXISTS`; add matching slide excerpt here for operators running DDL manually.

Example: partial index `CREATE INDEX ... ON result_cache (evaluation_id) WHERE evaluation_result IS NOT NULL`; for faster purge-by-quiz.

OCR-related module map (objective + utilities)

Module	Role
ocr_service.py	Legacy Gemini Flash stream per image.
optimized_ocr_service.py	/ Pooled regional endpoints + load scoring.
multi_region_ocr_service.py	
image_preprocessing.py	Paper isolation + JPEG shrink before POST.
pdf_split_service.py	Multi-page PDF → sheet-shaped dicts + stable hash IDs.
offline_ocr_service.py	Tesseract/Paddle/EasyOCR without cloud.
extract_text_from_images.py	Repo-root script for quick Vertex image tests.
api/batch_endpoints_optimized.py	Orchestrates dual OCR, cache, PDF runs.

Migration 001_result_cache.sql (core DDL)

```
CREATE TABLE IF NOT EXISTS public.result_cache (  
    file_hash          text PRIMARY KEY,  
    file_name          text NOT NULL,  
    ocr_result         jsonb NOT NULL,  
    evaluation_result  jsonb,  
    answer_key_hash    text,  
    created_at         timestampz NOT NULL DEFAULT now(),  
    last_accessed      timestampz NOT NULL DEFAULT now(),  
    access_count       integer NOT NULL DEFAULT 0,  
    result_size        integer NOT NULL DEFAULT 0  
);  
ALTER TABLE public.result_cache ENABLE ROW LEVEL SECURITY;
```

Migration 002_cache_evaluation_scope.sql

```
1 ALTER TABLE public.result_cache
2     ADD COLUMN IF NOT EXISTS evaluation_id text,
3     ADD COLUMN IF NOT EXISTS file_id      text;
4 CREATE INDEX IF NOT EXISTS idx_result_cache_eval_file
5     ON public.result_cache(evaluation_id, file_id);
```

Migration 002_pdf_page_map.sql

Repo uses two 002_*.sql files (evaluation scope vs PDF map); apply both when provisioning Supabase — order matches your migration runner or manual checklist.

```
CREATE TABLE IF NOT EXISTS public.pdf_page_map (  
  run_id      text NOT NULL,  
  pdf_hash    text NOT NULL,  
  page_number integer NOT NULL,  
  entry_number text,  
  name        text,  
  status      text NOT NULL,  
  total_score numeric,  
  max_score   numeric,  
  PRIMARY KEY (run_id, page_number)  
);
```

SQLite (database.py) vs Supabase

SQLite: `students`, `submissions`, `results`.

Supabase: `result_cache` (JSONB OCR + eval), scoped by `evaluation_id/file_id`; `pdf_page_map` for PDF reconciliation.

Cache PK: `sheet:v2:{evaluation_id}:{file_id}` consolidates OCR+eval rows.

multi_region_ocr_service vs optimized batch

Same **endpoint scoring idea** as in the **Throughput** section (frame titled *Load balancer score*): lower combined load + latency + model-priority + error-rate wins among healthy endpoints.

Important split: general OCR routing may prefer Flash/Lite pools; `batch_endpoints_optimized` fixes **Gemini 2.5 Pro** and adds token buckets + dual OCR + majority vote — see the throughput section for code and defaults.

optimized_ocr_service: tuning knobs

```
self.max_image_size = (1920, 1920)
self.jpeg_quality = 85
self.optimal_batch_size = 25
self.connection_pool_size = 500
```

Optimized batch OCR policy

Gemini 2.5 Pro only; dual independent OCR; disagree → third majority vote; token-bucket rate limiting; cache hit skips vision + scoring entirely (`batch_endpoints_optimized.py` module docstring).

Companion detail: token buckets, round-robin cooldowns, and pipeline semaphore defaults live in **Throughput, fuzzy matching, and HTTP contracts**.

image_preprocessing.preprocess_for_ocr

```
def preprocess_for_ocr(image_path: str,
                        max_size: Tuple[int, int] = (1920, 1920),
                        jpeg_quality: int = 85,
) -> Tuple[bytes, str]:
    """HSV_paper_crop->resize->JPEG_bytes."""
```

OCR and migrations: key takeaways

- **Migration order:** run 001 before 002. The two 002_*.sql files are independent and can be run in either order between themselves.
- **RLS is enabled but policies are minimal:** add restrictive policies before storing student-identifiable data in `result_cache`.
- **Preprocessor defaults:** 1920 px max and JPEG quality 85 balance checkbox legibility vs Vertex payload size limits (10 MB inline). Do not raise quality above 90 without testing payload sizes.
- **Offline OCR:** `offline_ocr_service.py` (Tesseract/Paddle/EasyOCR) is a drop-in replacement at the `extract_objective_sheet` interface level. Use it for air-gapped deployments or cost control.
- **New column pattern:** `ADD COLUMN IF NOT EXISTS` in migration + update `result_cache_service.py` upsert builder; add index if column appears in `WHERE` clauses.
- **Next:** Supplemental coverage documents the CLI, deployment configs, and test harnesses.

Supplemental coverage: what this section covers

This section documents the parts of the repo that are **not on the HTTP request path** but are essential for operators and developers: the headless CLI, deployment configuration, and existing test harnesses.

Area	What is here
Scoring internals	<code>evaluation_service.py</code> rules: SMCQ/MMCQ/NCQ, negative marking
Answer-key parsers	<code>answer_key_service.py</code> : CSV/XLSX/PDF/image routing
Drive service	Full responsibilities: recursive listing, MIME download, rename batching
Offline OCR	Local Tesseract/Paddle/EasyOCR engines
CLI headless pipeline	<code>cli.py</code> flags and flow
Deployment configs	<code>fly.toml</code> , <code>railway.toml</code>
Tests and utilities	Smoke tests, HEIC probes, auth fixtures
Fixture files	LeetCode case JSON samples for offline use

How to extend: supplemental area (CLI, deploy, tests)

Add a CLI flag: edit `backend/cli.py` — copy an existing `parser.add_argument(...)` block and branch in `main()` (mirroring `-local-ocr`).

Add a deploy secret: `fly secrets set KEY=value` or Railway variables dashboard — match names already read in `main.py/start.sh`.

Example (Fly): `fly secrets set NEW_FEATURE_FLAG=true` then redeploy; in Python call `os.getenv` with that same environment variable name.

Add a smoke check: duplicate `backend/smoke_test.py` pattern — HTTP GET `/api/status`, assert JSON keys.

evaluation_service.py: scoring rules

Pure `match_and_score(answer_key, student_answers)` — no LLM.

Normalizes student option tokens; walks each keyed question; supports SMCQ/MMCQ/NCQ with per-question positive/negative marks plus global negative marking fallback.

Treats X, NONE, NA style marks as unattempted (no negative).

Emits `QuestionResult` rows (marked, correct, result, score) aggregated into `StudentResult`.

Parallel-friendly wrappers around scoring (`batch_match_and_score`, optimized answer-key preprocessing) for folder/ZIP throughput — feeds bulk statistics surfaced in batch endpoints.

answer_key_service.py: formats

`extract_answer_key(path, mime)` routes CSV/XLSX/PDF/image parsers; vision path uses Gemini Flash stream via Vertex SA or API key URL when structured parse fails.
Normalizes into `AnswerKey` (answers dict of `AnswerKeyEntry`).
Shared by ZIP ingestion, Drive extracts, and manual uploads.

drive_service.py: responsibilities

Service-account OAuth primary; optional API key mode for public folders.

Folder ID extraction from URLs, recursive listing, MIME-aware downloads, batch rename orchestration, answer-key vs sheet separation heuristics (`separate_files`).

Batch rename batches HTTP calls with retry-friendly chunking (see `batch_rename_files`).

Local engines (**Tesseract**, **PaddleOCR**, **EasyOCR**) plus **ensemble** merge — output shape matches cloud `extract_objective_sheet` so `match_and_score` stays unchanged.
Heavy deps gated behind `requirements-local-ocr.txt`; lazy singletons amortize init cost.

cli.py: headless pipeline

```
python cli.py -drive-folder ... -sheet ...
```

Flags: -mode offline|online, -local-ocr ensemble|paddle|tesseract|easyocr, -tab-name, -no-export, dry-run options.

Flow: enumerate Drive images → offline or Gemini OCR → EvaluationService.match_and_score → optional SheetsService.update_marks.

Useful for CI batches without SPA + complements Docker-hosted API.

Deployment configs

fly.toml: Dockerfile build, internal port 8000, HTTPS, concurrency soft/hard limits, `/health` check, extended idle timeout for long OCR; comments document secrets (`SUPABASE_DB_URL`, base64 SA key).

railway.toml: Nixpacks builder; `pip install -r backend/requirements.txt`; start via `/app/backend/start.sh`.

Tests and utility scripts (repo root)

`backend/smoke_test.py`, `backend/test_phase3.py` — manual sanity harnesses.
`backend/test_heic_ocr.py`, `repo test_heic_ocr.py` — image-format probes.
`extract_text_from_images.py` — quick Vertex vision experiments.
`frontend-new-design/test_supabase_auth.js`, `create_test_user.js` — auth fixtures.
Not wired into unified CI in this deck — document presence for onboarding QA.

Sample artifacts for offline exam bundles:

leetcode_cases_sample.json, testcases_sample.json, testcases_sample.txt, test_expected.txt, test_stdin.txt — illustrate JSON case format consumed by leetcode_mode without hitting live GraphQL.

Pair with cases_file resolver strategy documented earlier.

Supplemental coverage: key takeaways

- **CLI is fully headless:** `backend/cli.py` reproduces the Drive → OCR → score → Sheets flow without a running API server. Useful for CI/CD batch jobs.
- **Testing is manual today:** `smoke_test.py` and `test_phase3.py` are run-by-hand harnesses. There is no automated CI pipeline in the repo yet — a natural extension is adding GitHub Actions to run the smoke test on every push.
- **Deploy secrets live outside code:** Fly secrets and Railway environment variables are set via CLI or dashboard. Never commit SA keys or Supabase URLs to the repo.
- **Fixture files are your offline safety net:** ship `leetcode_cases_sample.json`-style files alongside exam ZIPs in any air-gapped exam environment to avoid LeetCode GraphQL dependency.
- **HEIC support:** `test_heic_ocr.py` probes whether the OCR pipeline can handle iPhone photos. Ensure Pillow + HEIC plugin are installed on the grading server if students submit HEIC images.
- **Next:** HTTP signatures documents the exact REST shapes for Vertex AI, Drive API v3, and Sheets API v4 — useful when debugging raw network traffic.

HTTP signatures: what this section covers

This section is a **reference for the three external APIs** this repo calls. Use it when reading raw network traffic in a proxy, debugging unexpected 4xx errors, or writing a new call site without the Python client library.

API	Documented operations
Vertex AI / Gemini	<code>streamGenerateContent</code> (Vertex SA bearer + AI Studio key variants)
Google Drive API v3	List folder, download binary, export Google file, rename (PATCH)
Google Sheets API v4	Spreadsheet metadata, read grid, write grid, batchUpdate (structural ops)

All three APIs require an OAuth bearer token (service-account) or API key. The Python client libraries (`google-api-python-client`, `google-auth`) handle token refresh automatically; this section shows the underlying HTTP so you understand what the libraries are doing.

Vertex AI (Gemini): REST signature (ocr_service, batch OCR)

Endpoints (repo picks one transport):

Service account / Vertex: POST

<https://{region}-aiplatform.googleapis.com/v1/projects/{project}/locations/{region}/publishers/google/models/{model}:streamGenerateContent>

API key (AI Studio style): POST <https://generativelanguage.googleapis.com/v1beta/models/{model}:streamGenerateContent?key=...>

Request JSON (vision OCR): Authorization: Bearer <token> when using Vertex SA; body shape:

```
{
  "contents": [{
    "role": "user",
    "parts": [
      {"text": "..."},
      {"inline_data": {"mime_type": "image/jpeg", "data": "BASE64..."}}
    ]
  }]
}
```

Response: HTTP 200 JSON; streaming may be a JSON array of chunks or one object. Parsed body exposes `candidates[].content.parts[].text`

(concatenate); application code treats that text as model output (JSON sheet extraction).

Google Drive API v3: REST signature (drive_service)

Base URL <https://www.googleapis.com/drive/v3>. OAuth scope used: <https://www.googleapis.com/auth/drive>. Python client maps to HTTP below.

List folder: GET /files?q='{folderId}' in parents and
trashed=false&pageSize=100&fields=nextPageToken,files(id,name,mimeType,webContentLink,size)&supportsAllDrives=true
→ { "files": [{ "id", "name", "mimeType", ... }], "nextPageToken": ... }

Download binary: GET /files/{fileId}?alt=media → raw bytes (chunked download in client).

Export Google file: GET /files/{fileId}/export?mimeType=text/csv (or PDF, etc.) → export bytes.

Rename file: PATCH /files/{fileId}?supportsAllDrives=true with body {"name": "New Name.ext"} → updated file metadata JSON.

Google Sheets API v4: REST signature (sheets_service)

Base URL <https://sheets.googleapis.com/v4/spreadsheets>. Scope: <https://www.googleapis.com/auth/spreadsheets>.

Spreadsheet metadata: GET `/spreadsheetId` → { "spreadsheetId", "sheets": [{ "properties": { "title", "sheetId", ... } }] }

Read grid: GET `/spreadsheetId/values/{A1_range_or_sheet_name}` → { "range", "majorDimension": "ROWS", "values": [["hdr",...], [...]] }

Write grid: PUT `/spreadsheetId/values/{range}?valueInputOption=USER_ENTERED` with body { "values": [...] } → { "updatedCells", "updatedRows", ... }

Structural ops: POST `/spreadsheetId:batchUpdate` with { "requests": [...] } (add sheet, merge, repeatCell, clear) → { "replies", ... }.

HTTP signatures: key takeaways

- **Vertex vs AI Studio:** both use the same JSON body shape; the only difference is the URL pattern and auth header. Use Vertex in production (quota is project-level); AI Studio key for quick experiments.
- **Drive download requires alt=media:** omitting the query param returns metadata JSON instead of file bytes — a very common mistake when writing new download code.
- **Drive rename is a PATCH:** only send `{"name": "..."} in the body; include supportsAllDrives=true or the call silently fails on shared drives.`
- **Sheets valueInputOption=USER_ENTERED:** use this so formulas (like the SUM in the statistics block) are evaluated. RAW stores the literal string `=SUM(...)`.
- **batchUpdate for structure:** adding a new tab, merging cells, or applying formatting all go through `batchUpdate` with a `requests` array. Keep structural changes separate from data writes (separate API calls) to simplify error handling.

Troubleshooting: this section covers common failure modes

This is a reference for the most frequently encountered problems when running or extending the project. Each entry includes the symptom you observe, the root cause, and the concrete fix.

Issues are grouped by subsystem: **startup/environment** → **OCR/Vertex** → **cache/Supabase** → **code-eval** → **Sheets/Drive** → **frontend**.

Troubleshooting: startup and environment

Symptom	Root cause	Fix
/api/status returns 502	Backend not running	uvicorn main:app --reload --port 8000 from backend/
Backend starts but cache shows sqlite despite CACHE_BACKEND=supabase	Pool connection failed at startup	Check SUPABASE_DB_URL format; run migrations; check firewall to Supabase host
CORS error in browser console	CORS_ORIGINS does not include frontend URL	Add http://localhost:5173 (or deployed domain) to CORS_ORIGINS env var
ModuleNotFoundError: fitz	PyMuPDF not installed	pip install pymupdf in backend virtualenv
Frontend shows blank page after login	VITE_BACKEND_URL has trailing slash	Remove trailing slash from .env.local
storageKey mismatch / auto-logout	Wrong PROJECT_REF in supabaseClient.js	Verify project ref matches your Supabase project URL slug

Troubleshooting: OCR and Vertex

Symptom	Root cause	Fix
All OCR results show empty answers	SA JSON path wrong or missing	Verify <code>GOOGLE_APPLICATION_CREDENTIALS</code> is an absolute path to a valid JSON key
429 Too Many Requests storms	Vertex RPM quota exceeded	Lower <code>PIPELINE_MAX_CONCURRENT</code> ; add more regions to <code>_ENDPOINT_POOL</code>
OCR works for 5 sheets then hangs	429 cooldown (30s–300s per region)	Expected behavior; cooldowns auto-clear. Add regions to expand throughput.
<code>google.auth.exceptions.DefaultCredentialsError</code>	No credentials in environment	Set <code>GOOGLE_APPLICATION_CREDENTIALS</code> or <code>GEMINI_API_KEY</code>
Image preprocessor returns zero bytes	HSV paper-crop clips the whole image	Reduce <code>contrast</code> <code>threshold</code> in <code>image_preprocessing.preprocess_for_ocr</code>
Stale results after editing answer key	Cache rows hold old <code>answer_key_hash</code>	<code>DELETE /api/cache/purge-evaluation/{evaluation_id}</code> or restart (auto-purge on lifespan)

Troubleshooting: code evaluation

Symptom	Root cause	Fix
All code results <code>COMPILE_ERROR</code>	gcc not on PATH	Install build-essential (Linux) or Xcode CLI tools (macOS); verify <code>gcc -version</code>
<code>HARNESS_BUILD_ERROR</code> on every case	Student function signature does not match LeetCode spec	Check <code>selection_notes</code> in result; may need <code>function_name_override</code> in answer key
LeetCode resolver returns 0 cases	GraphQL blocked by firewall	Ship a <code>cases_file</code> JSON alongside the exam ZIP (see <code>_parse_cases_file</code> slide)
Repair agent rewrites student logic	<code>validate_trivial_fix</code> not catching the change	Do not disable the validator; instead report the issue and examine the diff in result comments
<code>TIMEOUT</code> on all test cases	Student code has infinite loop	Expected; timeout is set per binary run. Report as-is; do not extend the timeout.
Code-eval endpoint ignores SA key	<code>vertex_key.json</code> not in <code>c_code_evaluator/</code>	Place the SA key JSON at that path or set <code>GOOGLE_APPLICATION_CREDENTIALS</code> globally

Troubleshooting: Sheets, Drive, and frontend

Symptom	Root cause	Fix
Export writes zero rows	No results in session or SQLite; answer key not loaded	Load answer key first; re-run grading pipeline before exporting
Sheets export shows all students UNMATCHED	Entry-number pattern does not match institution format	Edit <code>ENTRY_NUMBER_PATTERN</code> regex in <code>sheets_service.py</code>
Drive rename produces <code>HttpError 403</code>	SA key lacks drive scope	Recreate SA key with <code>https://www.googleapis.com/auth/drive scope</code>
Super Sheet column shifts incorrectly	Tab named differently from "Super Sheet"	The tab name is hardcoded; rename the tab in Sheets to match exactly
Frontend drive folder URL gives 400	URL not parseable as folder ID	Paste the URL from Drive share dialog, not from browser address bar
<code>evaluation_id</code> 400 error	"default" or blank passed from frontend	Ensure <code>evaluationId</code> is loaded from <code>useParams()</code> before calling batch endpoints

Quick reference: most-used files and endpoints

What you need	Where to find it
Key backend files	
Add endpoint (standard)	backend/api/endpoints.py + backend/models.py
Add endpoint (batch)	backend/api/batch_endpoints_optimized.py
Add frontend page	frontend-new-design/src/pages/ + Routes.jsx + backendService.js
Scoring rules	backend/services/evaluation_service.py: match_and_score
OCR pipeline	backend/services/optimized_ocr_service.py / multi_region_ocr_service.py
Fuzzy matching	backend/services/sheets_service.py: _smart_match_score
Rename policy	backend/services/drive_rename_service.py
Essential endpoints	
Health check	GET /api/status
Load answer key (no file)	POST /api/answer-key/set-manual
Batch Drive grading	POST /api/batch/process-folder-optimized
Check batch progress	GET /api/batch/processing-status/{id}
Export to Sheets	POST /api/export-to-sheets
Purge stale cache	DELETE /api/cache/purge-evaluation/{evaluation_id}
Key env vars	
Google credentials	GOOGLE_APPLICATION_CREDENTIALS, GOOGLE_CLOUD_PROJECT
Vertex region	VERTEX_LOCATION (default: us-central1)
Cache backend	CACHE_BACKEND=supabase + SUPABASE_DB_URL
RAM/throughput knob	PIPELINE_MAX_CONCURRENT (default 24)
Frontend → backend	VITE_BACKEND_URL (no trailing slash)

Thank you